

I/O 操作的实现

在第 1 章介绍冯·诺依曼结构计算机时，提到输入/输出子系统（I/O 子系统）是计算机的重要组成部分。无论是在应用程序员编写用户程序时，还是在最终用户使用操作系统提供的人机交互功能使用计算机时，都会涉及如何让外部设备进行 I/O 的问题。

使用高级语言编写应用程序时，通常利用编译系统提供的专门的 I/O 库函数来实现外设的 I/O 功能，而 I/O 库函数通常将具体的 I/O 操作功能通过相应的陷阱指令（自陷指令，有时也称“软中断指令”）以“系统调用”的方式转换为由操作系统内核来实现。也就是说，任何 I/O 操作过程最终都是由操作系统内核控制完成的。

本章主要介绍与 I/O 操作相关的软件和硬件方面的相关内容。主要包括：文件的概念，与 I/O 系统调用相关的函数，基本的 C 标准 I/O 库函数，常用外设控制器（I/O 接口）的基本功能和结构，I/O 端口的编址方式，外设与主机之间的 I/O 控制方式以及如何利用陷阱指令将用户 I/O 请求转换为操作系统的 I/O 处理过程。

8.1 I/O 子系统概述

I/O 子系统主要解决各种形式信息的输入和输出问题，即解决如何将所需信息（文字、图表、声音、视频等）通过不同外设输入到计算机中，或者计算机内部处理的结果如何通过相应外设输出给用户。

所有高级语言的运行时系统都提供了执行 I/O 功能的高级机制，例如，C 语言中提供了像 `printf()` 和 `scanf()` 等这样的标准 I/O 库函数，C++ 语言中提供了如“<<”（输入）和“>>”（输出）这样的重载 I/O 操作符。从用户在高级语言程序中通过 I/O 函数或 I/O 操作符提出 I/O 请求，到 I/O 设备响应并完成 I/O 请求，整个过程涉及多个层次的 I/O 软件和 I/O 硬件的协调工作。

小贴士

运行时系统（runtime system）也称为运行时环境（runtime environment）或简称为运行时（runtime），它实现了一种计算机语言的核心行为。不管是被编译转换的语言，还是被解释执行的语言，或者是嵌入式领域特定的语言等，每一种计算机语言都实现了某种形

式的运行时系统。一个运行时系统除了要支持语言基本的低级行为之外，还要实现更高层次的行为，如库函数等，甚至提供类型检查、调试以及代码生成与优化等功能。

与计算机系统一样，I/O 子系统也采用层次结构。图 8.1 是 I/O 子系统层次结构示意图。

I/O 子系统包含 I/O 软件和 I/O 硬件两大部分。I/O 软件包括最上层提出 I/O 请求的用户空间 I/O 软件（称为用户 I/O 软件）和在底层操作系统中对 I/O 进行具体管理和控制的内核空间 I/O 软件（称为系统 I/O 软件），系统 I/O 软件又分三个层次，分别是与设备无关的 I/O 软件层、设备驱动程序层和中断服务程序层。I/O 硬件在操作系统内核空间 I/O 软件的控制下完成具体的 I/O 操作。

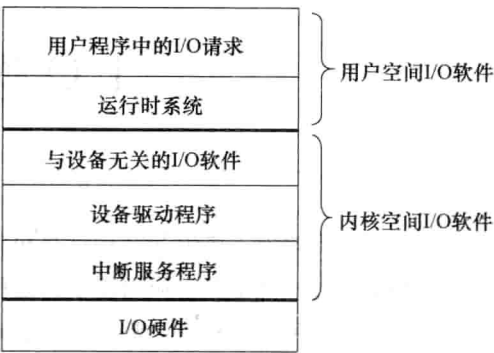


图 8.1 I/O 子系统层次结构

操作系统在 I/O 子系统中承担极其重要的作用，这主要是由 I/O 子系统的以下三个特性决定的。

① 共享性。I/O 子系统被多个进程共享，因此必须由操作系统对共享的 I/O 资源统一调度管理，以保证用户程序只能访问自己有权访问的那部分 I/O 设备或文件，并使系统的吞吐率达到最佳。

② 复杂性。I/O 设备控制的细节比较复杂，如果由最上层的用户程序直接控制，则会给广大的应用程序开发者带来麻烦，因而需操作系统提供专门的驱动程序进行控制，这样可以对应用程序员屏蔽设备控制的细节，简化应用程序开发。

③ 异步性。I/O 子系统的速度较慢，而且不同设备之间的速度也相差较大，因而，I/O 设备与主机之间的信息交换方式通常使用异步的中断 I/O 方式。中断导致从用户态向内核态转移，因此，I/O 处理须在内核态完成，通常由操作系统提供中断服务程序来处理 I/O。

用户程序总是通过某种 I/O 函数或 I/O 操作符请求 I/O 操作。例如，用户程序需要读一个磁盘文件中的记录时，它可以通过调用 C 语言标准 I/O 库函数 `fread()`，也可以直接调用 `read` 系统调用的封装函数 `read()` 来提出 I/O 请求。不管用户程序中调用的是 C 库函数还是系统调用封装函数，最终都是通过操作系统内核提供的系统调用来实现 I/O。图 8.2 给出了用户程序调用 `printf()` 来调出内核提供的 `write` 系统调用的过程。

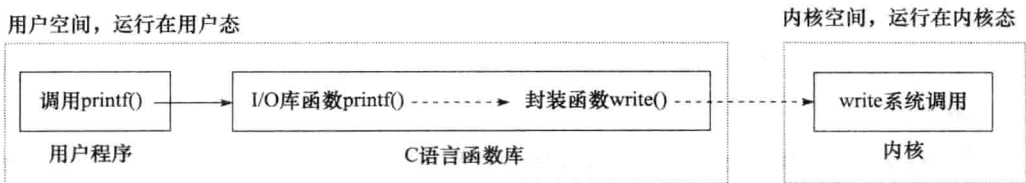


图 8.2 用户程序、C 语言库和内核之间的关系

从图 8.2 可以看出，对于一个 C 语言用户程序，若在某过程（函数）中调用了 `printf()`，则

在执行到调用 `printf()` 的语句时,便会转到 C 语言函数库中对应的 I/O 标准库函数 `printf()` 去执行,而 `printf()` 最终又会转到调用函数 `write()`;在执行到调用 `write()` 的语句时,便会通过一系列步骤在内核空间中找到 `write` 对应的系统调用服务例程来执行,从而从用户态转到内核态执行。

每个系统调用的封装函数会被转换为一组与具体机器架构相关的指令序列,这个指令序列中至少有一条陷阱指令,在陷阱指令之前可能还有若干条传送指令用于将 I/O 操作的参数送入相应的寄存器。

例如,在 IA-32 中,陷阱指令就是 `INT n` 指令,也称为软中断指令。在早期 IA-32 架构中,Linux 系统将 `int $0x80` 指令用作系统调用,在系统调用指令之前会有一串传送指令,用来将系统调用号等参数传送到相应的寄存器。系统调用号通常在 `EAX` 寄存器中,可根据系统调用号选择执行一个系统调用服务例程。用户进程的 I/O 请求通过调出操作系统中相应的系统调用服务例程来实现。

I/O 子系统工作的大致过程如下:首先,CPU 在用户态执行用户进程,当 CPU 执行到系统调用封装函数对应的指令序列中的陷阱指令时,会从用户态陷入到内核态;转到内核态执行后,CPU 根据陷阱指令执行时 `EAX` 寄存器中的系统调用号,选择执行一个相应的系统调用服务例程;在系统调用服务例程的执行过程中可能需要调用具体设备的驱动程序;在设备驱动程序执行过程中启动外设工作,外设准备好后发出中断请求,CPU 响应中断后,就调出中断服务程序执行,在中断服务程序中控制主机与设备进行具体的数据交换。

图 8.3 是 Linux 系统中 `write` 操作的执行过程示意图。

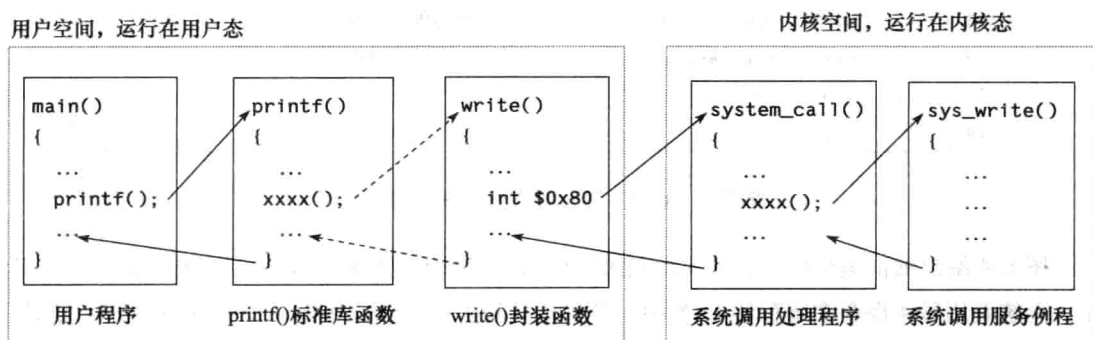


图 8.3 在 Linux 系统中 `write` 操作的执行过程

如图 8.3 所示,假定用户程序中有一个语句调用了库函数 `printf()`,在 `printf()` 函数中又通过一系列的函数调用,最终转到调用 `write()` 函数。在 `write()` 函数对应的指令序列中,一定有一条用于系统调用的陷阱指令,在 IA-32/Linux 系统中就是指令 `int $0x80` 或 `sysenter`。该陷阱指令执行后,进程就从用户态陷入到内核态执行。Linux 中有一个系统调用的统一入口,即系统调用处理程序 `system_call()`。CPU 执行陷阱指令后,便转到 `system_call()` 的第一条指令执行。在 `system_call()` 中,将根据 `EAX` 寄存器中的系统调用号跳转到当前系统调用对应的系统调用服务例程 `sys_write()` 去执行。`system_call()` 执行结束时,从内核态返回到用户态下的陷阱指令后面一条指令继续执行。

Linux 系统下 `write()` 函数的用法如下:

```
ssize_t write(int fd, const void *buf, size_t n);
```

这里的类型 `size_t` 和 `ssize_t` 分别是 `unsigned int` 和 `int`。字节数 n 通常是 `unsigned` 类型，但是，因为返回值还可能是 -1 ，所以，返回类型只能是带符号整数类型 `int`。

`write()` 封装函数的源代码编译后生成如图 8.4 所示的汇编代码。按照函数调用时压栈的顺序可知，在某函数中调用 `write()` 函数时，最先被压入栈中的参数是 n ，其次是 buf ，最后是 fd ，参数压栈后执行调用指令 `call`，此时，再将返回地址压栈。在执行完 `call` 指令后，便跳转到图 8.4 所示的 `write` 过程执行。执行第 2 行的 `pushl` 指令后，当前栈指针寄存器内容 `R[esp]` 指向刚保存的 `R[ebx]`，`R[esp] + 4` 指向返回地址，`R[esp] + 8` 指向参数 fd ，`R[esp] + 12` 指向参数 buf ，`R[esp] + 16` 指向参数 n 。

```

1  write:
2      pushl  %ebx           // 将EBX入栈
3      movl   $4, %eax       // 将系统调用号送EAX
4      movl   8(%esp), %ebx   // 将第1个参数fd送EBX
5      movl   12(%esp), %ecx  // 将第2个参数buf送ECX
6      movl   16(%esp), %edx  // 将第3个参数n送EDX
7      int    $0x80         // 进入系统调用处理程序system_call执行
8      cmpl   $-132, %eax    // 检查返回值，假定最大出错码为131
9      jbe    .L1           // 若无错误，则跳转至.L1
10     negl   %eax           // 将返回值取负送EAX
11     movl   %eax, error    // 将EAX的值送error
12     movl   $-1, %eax      // 将write函数返回值置-1
13  .L1:  popl   %ebx
14      ret

```

图 8.4 `write()` 封装函数对应的汇编代码

图 8.4 给出的汇编代码中，第 3 行到第 6 行用来将系统调用的参数送到不同的寄存器，其中，系统调用号 4 保存在寄存器 `EAX` 中。第 7 行是陷阱指令 `int $0x80`，CPU 执行到该指令时，将从用户态切换到内核态，调出系统调用处理程序 `system_call()` 执行。在 `system_call()` 中，根据系统调用号为 4，再跳转到相应的系统调用服务例程 `sys_write()` 执行，以完成将一个字符串写入文件的功能，其中，字符串的首地址由 `ECX` 指定，字符串的长度由 `EDX` 指出，写入文件的文件描述符由 `EBX` 指出。`system_call()` 执行结束时，从内核返回的参数存放在 `EAX` 中。若返回参数表明在内核中执行系统调用发生错误，则将 `EAX` 取负后得到错误码，存放在 `error` 中，并将 `write()` 函数的返回值置 -1 ；若没有发生错误，则 `write()` 函数的返回值就是从内核系统调用返回的值，它通常是真正写入文件的字节数。

8.2 用户空间 I/O 软件

I/O 软件包括图 8.1 所示的最上层提出 I/O 请求的用户空间 I/O 软件和在底层操作系统中

进行 I/O 具体控制的内核空间 I/O 软件。

8.2.1 用户程序中的 I/O 函数

在用户空间 I/O 软件中, 用户程序可以通过调用特定的 I/O 函数提出 I/O 请求。在 UNIX/Linux 系统中, 用户程序使用的 I/O 函数可以是 C 标准 I/O 库函数或系统调用的封装函数, 前者如文件 I/O 函数 `fopen()`、`fread()`、`fwrite()` 和 `fclose()` 或控制台 I/O 函数 `printf()`、`putc()`、`scanf()` 和 `getc()` 等, 后者如 `open()`、`read()`、`write()` 和 `close()` 等。

标准 I/O 库函数比系统调用的封装函数抽象层次高, 后者属于 系统级 I/O 函数, 前者是基于后者实现的。图 8.5 给出了两者之间的关系。

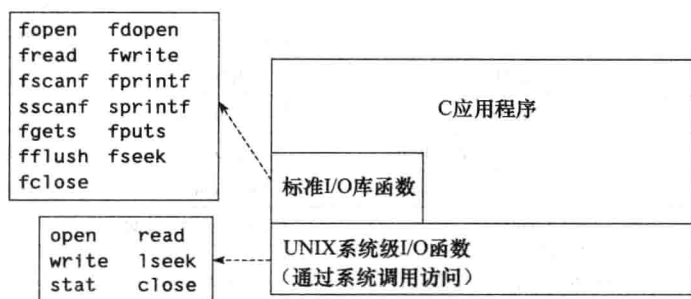


图 8.5 C 标准 I/O 库函数与 UNIX 系统级 I/O 函数之间的关系

通常情况下, C 程序员大多使用较高层次的标准 I/O 库函数, 而很少使用底层的系统级 I/O 函数。使用标准 I/O 库函数得到的程序移植性较好, 可以在不同体系结构和操作系统平台下运行。但是, 使用 C 标准 I/O 库函数有以下不足: ① 不存在支持文件安全性的函数, 因而不能保证文件的安全性; ② 所有 I/O 都是同步的, 即程序必须等待 I/O 操作真正完成后才能继续执行; ③ 在一些情况下不适合甚至无法使用标准 I/O 库函数实现 I/O 功能, 例如, C 标准 I/O 库中不提供读取文件元数据的函数; ④ 更有甚者, 标准 I/O 库函数还存在一些问题, 使得用它进行网络编程会造成易于出现缓冲区溢出等风险, 同时它也不提供对文件进行加锁和解锁等功能。

虽然在很多情况下使用标准 I/O 库函数就能解决问题, 特别是对于磁盘和终端设备 (键盘、显示器等) 的 I/O 操作。但是, 在 UNIX/Linux 系统中, 有时用标准 I/O 库函数或系统级 I/O 函数对网络设备进行 I/O 操作时会出现一些问题, 因此, 也可以基于底层的系统级 I/O 函数自行构造高层次 I/O 函数, 以提供适合网络 I/O 的读操作和写操作函数。

在 Windows 系统中, 用户程序同样可以调用 C 标准 I/O 库函数, 此外, 还可以调用 Windows 提供的 API 函数, 如文件 I/O 函数 `CreateFile()`、`ReadFile()`、`WriteFile()`、`CloseHandle()` 和控制台 I/O 函数 `ReadConsole()`、`WriteConsole()` 等。

表 8.1 给出了关于文件 I/O 和控制台 I/O 的部分函数对照列表, 其中包含了 C 标准 I/O 库函数、UNIX/Linux 系统级 I/O 函数和用于 I/O 的 Windows API 函数。

表 8.1 关于 I/O 操作的部分函数对照表

序号	C 标准库	UNIX/Linux	Windows	功能描述
1	getc, scanf, gets	read	ReadConsole	从标准输入读取信息
2	fread	read	ReadFile	从文件读入信息
3	putc, printf, puts	write	WriteConsole	在标准输出上写信息
4	fwrite	write	WriteFile	在文件上写入信息
5	fopen	open, creat	CreateFile	打开/创建一个文件
6	fclose	close	CloseHandle	关闭一个文件 (CloseHandle 不限于文件)
7	fseek	lseek	SetFilePointer	设置文件读写位置
8	rewind	lseek (0)	SetFilePointer (0)	将文件指针设置成指向文件开头
9	remove	unlink	DeleteFile	删除文件
10	feof	无对应	无对应	停留到文件末尾
11	perror	strerror	FormatMessage	输出错误信息
12	无对应	stat, fstat, lstat	GetFileTime	获取文件的时间属性
13	无对应	stat, fstat, lstat	GetFileSize	获取文件的长度属性
14	无对应	fent	LockFile/UnlockFile	文件的加锁、解锁
15	使用 stdin、stdout 和 stderr	使用文件描述符 0、1 和 2	GetStdHandle	标准输入、标准输出和标准错误设备

从表 8.1 可以看出, C 标准库中提供的函数并没有涵盖所有底层操作系统提供的功能, 如表中第 12、13 和第 14 项; 不同的 C 标准库函数可能调用相同的系统调用, 例如, 表中第 1 和第 2 项中不同的 C 库函数是由同一个系统调用 read 实现的, 同样, 表中第 3 和第 4 项中不同的 C 库函数都是由 write 系统调用实现的; 此外, C 标准 I/O 库函数、UNIX/Linux 和 Windows 的 API 函数所提供的 I/O 操作功能并不是一一对应的。虽然对于基本的 I/O 操作, 它们有大致一样的功能, 不过, 在使用时还是要注意它们之间的不同。其中一个重要的不同点是, 它们的参数中对文件的标识方式不同, 例如, 函数 read() 和 write() 的参数中指定的文件用一个整数类型的文件描述符来标识; 而 C 标准库函数 fread() 和 fwrite() 的参数中指定的文件用一个指向特定结构的指针类型来标识。下一节简单介绍与文件相关的基本概念, 而有关文件系统的细节内容请参看操作系统方面的书籍。

8.2.2 文件的基本概念

Linux 操作系统是一个类 UNIX 系统, 其文件格式和有关文件操作方面的系统调用等与 UNIX 中的类似。在 UNIX 系统中, 所有的 I/O 操作都是通过读写一个文件来实现的, 所有外设, 包括网络 (套接字 socket)、终端设备 (键盘和显示器) 等, 都被看成是一个文件。把所有不同的物理设备抽象成一个逻辑上统一的“文件”后, 使得对于用户程序来说, 访问一个物理设备与访问一个真正的磁盘文件是完全一致的, 这样, 就为用户程序和外设之间的信息交换提供了一个统一的处理接口。

在 UNIX 操作系统中, 文件就是一个字节序列, 因此, 键盘可被看成是可以读取字节序列的输入设备文件, 显示器看成是可以写入字节序列的输出设备文件, 网络套接字是可以读取字

节序列和写入字节序列的输入/输出设备文件。通常将键盘和显示器构成的设备称为终端 (terminal)，对应标准输入文件和标准输出文件。像磁盘、光盘等外存储器上的文件则是普通文件。

根据文件中的每个字节是否是可读的 ASCII 码，可将文件分成ASCII 文件和二进制文件两类。ASCII 文件也称为文本文件，可以由多个正文行组成，每行以换行符 ('\\n') 结束，其中每个字节是一个字符。通常，终端设备上的标准输入文件和标准输出文件是 ASCII 文件；磁盘上的普通文件则可能是文本文件或二进制文件，例如，磁盘上的可重定位目标文件和可执行文件都是二进制文件，而源程序文件则是 ASCII 文件。

对于系统中的文件，用户程序可以对其进行创建、打开、读写和关闭等操作。

1. 创建文件

在大多数情况下，在读或写一个文件之前，用户程序必须告知系统将要对该文件进行什么操作，是读、写、添加还是可读可写，这个告知操作通过打开一个文件或创建一个文件来实现。

对于一个已存在的文件，可以直接打开文件；对于一个不存在的文件，则应先创建。创建一个新文件时，用户应指定所创建文件的文件名和访问权限，系统返回一个非负整数，它被称为文件描述符 (file descriptor)。文件描述符用于标识被创建的文件，在以后对文件的读写等操作时用文件描述符代表文件。

2. 打开文件

打开文件时，系统要检测文件是否存在、用户是否有访问文件的权限等。若这些操作都没有问题，则系统会返回一个非负整数作为文件描述符。

在 UNIX 中创建每个进程时，都会自动打开三个标准文件：标准输入（描述符为 0）、标准输出（描述符为 1）和标准错误（描述符为 2）。键盘和显示器可以分别抽象成标准输入文件和标准输出文件。

3. 设置文件读写位置

每个文件都有一个当前读写位置，它表示相对于文件最开始的字节偏移量，初始时为 0。用户程序中可以用 lseek 系统调用设置文件读写位置。

4. 读文件和写文件

一个新文件被创建后，用户程序可以将信息写到文件上。一个已存在的文件被打开后，用户程序可以从文件中读或写信息。写文件的操作从当前读写位置 $k(k \geq 0)$ 处写入 $n(n > 0)$ 个字节，因而写入后文件当前读写位置为 $k + n$ 。

读文件的操作从文件当前读写位置 $k(k \geq 0)$ 处读出 $n(n > 0)$ 个字节，因而读出后文件当前读写位置为 $k + n$ 。假设文件大小为 m 个字节，当执行读文件操作时，若 $k = m$ ，则当前位置为结尾处，这种情况称为文件结束 EOF (end of file)。

5. 关闭文件

当完成对文件的读写等操作后，用户程序需要通知内核关闭文件，表示用户程序不再对文件进行任何操作。关闭文件时将释放文件创建或打开时所创建的数据结构所在存储区，并回收文件描述符。不管一个进程因为何种原因终止，内核都会关闭所打开的文件，以释放其所用的存储资源。

8.2.3 系统级 I/O 函数

前面提到，与 I/O 操作相关的系统调用的封装函数属于系统级 I/O 函数。在 UNIX/Linux 系统中，常用的这类函数有 `creat()`、`open()`、`read()`、`write()`、`lseek()`、`stat()`/`fstat()`、`close()` 等，它们的调用形式及其功能说明如下。使用以下函数时必须包含相应的头文件（如 `unistd.h` 等）。

1. creat 系统调用

用法：`int creat(char *name, mode_t perms);`

第一个参数 *name* 为需创建的新文件的名称，是一个表示路径名和文件名的字符串；第二个参数 *perms* 用于指定所创建文件的访问权限，共有 9 位，分别指定文件拥有者、拥有者所在组成员以及其他用户各自所拥有的读、写和执行权限。通常用一个 8 进制数字中的三位分别表示读、写和执行权限，例如，`perms = 0755`，表示拥有者具有读、写和执行权限，而拥有者所在组成员和其他用户都只有读和执行权限。正常情况下，该函数返回一个文件描述符，若出错，则返回 -1。若文件已经存在，则该函数将把文件截断为长度为 0 的文件，也即，将文件原先的内容全部丢弃，因此，创建一个已存在的文件不会发生错误。

2. open 系统调用

用法：`int open(char *name, int flags, mode_t perms);`

除了默认的标准输入、标准输出和标准错误三种文件是自动打开以外，其他文件必须用相应的函数显式创建或打开后才能读写，可以用 `open` 系统调用显式打开文件。

正常情况下，`open()` 函数返回一个文件描述符，它是一个用以唯一标识被打开文件的非负整数，若出错，则返回 -1。第一个参数 *name* 为需打开文件的名称，是一个表示路径名和文件名的字符串；第二个参数 *flags* 指出用户程序将会如何访问这个打开文件，例如：

`O_RDONLY`：只读。

`O_WRONLY`：只写。

`O_RDWR`：可读可写。

`O_WRONLY | O_APPEND`：可在文件末尾添加并且只写。

`O_RDWR | O_CREAT`：若文件不存在则创建一个空文件并且可读可写。

`O_WRONLY | O_CREAT | O_TRUNC`：若文件不存在则创建一个空文件，若文件存在则截断为空文件，并且可读可写。

上述这些带 `O_` 的常数必须事先定义在某个头文件中，例如，在 System V UNIX 系统的头文件 `fcntl.h` 或 BSD 版本的头文件 `sys/file.h` 中都定义了这些常数。

假定用户程序将以只读方式使用文件 `test.txt`，则可以用以下语句打开文件：

```
fd = open("test.txt", O_RDONLY, 0);
```

第三个参数 *perms* 用于指定所创建文件的访问权限，通常在 `open()` 函数中该参数总是 0，除非以创建方式打开，此时，参数 *flags* 中应带有 `O_CREAT` 标志。不以创建方式打开一个文件时，若文件不存在，则发生错误。对于不存在的文件，可用 `creat` 系统调用来打开。

3. read 系统调用

用法：`ssize_t read(int fd, void *buf, size_t n);`

该函数功能是将文件 *fd* 中从当前读写位置 *k* 开始读取 *n* 个字节到 *buf* 中, 读操作后文件当前读写位置为 $k+n$ 。假定文件长度为 *m*, 当 $k+n > m$ 时, 则真正读取的字节数为 $m-k < n$, 并且读操作后文件当前读写位置为文件尾。函数返回值为实际读取字节数, 因而, 当 $m = k$ (EOF) 时, 返回值为 0; 出错时返回值为 -1。

4. write 系统调用

用法: `ssize_t write(int fd, const void *buf, size_t n);`

该函数功能是将 *buf* 中的 *n* 字节写到文件 *fd* 中, 从当前读写位置 *k* 处开始写入。返回值为实际写入字节数 *m*, 写入后文件当前读写位置为 $k+m$ 。对于普通的磁盘文件, 实际写入字节数 *m* 等于指定写入字节数 *n*。出错时返回值为 -1。

对于 read 和 write 系统调用, 可以一次读 (写) 任意字节, 例如, 每次读 (写) 一个字节或一个物理块大小, 如一个磁盘扇区 (512 字节) 或一个记录大小等。显然, 按照一个物理块大小来读 (写) 比较好, 可以减少系统调用的次数。

有些情况下, read 和 write 真正读 (写) 的字节数比用户程序设定的所需字节数要少, 这种情况并不被看成是一种错误。通常, 在读 (写) 磁盘文件时, 除非遇到 EOF, 否则不会出现这种情况。但是, 当读写的是终端设备文件、网络套接字文件、UNIX 管道、Web 服务器等时, 都可能出现这种情况。

5. lseek 系统调用

用法: `long lseek(int fd, long offset, int origin);`

当随机读写一个文件的信息时, 当前读写位置可能并非正好是马上要读或写的位置, 此时, 需要用 lseek() 函数来调整文件的当前位置。第一个参数 *fd* 指出需调整位置的文件; 第二个参数 *offset* 指出相对字节数; 第三个参数 *origin* 指出 *offset* 相对的基准, 可以是文件开头 (*origin* = 0)、当前位置 (*origin* = 1) 和文件末尾 (*origin* = 2)。例如:

```
lseek(fd, 0L, 2);    // 定位到文件末尾
lseek(fd, 0L, 0);    // 定位到文件开始
```

函数的返回值就是新的位置值, 若发生错误, 则返回 -1。

6. stat/fstat 系统调用

用法: `int stat(const *name, struct stat *buf);`

`int fstat(int fd, struct stat *buf);`

文件的所有属性信息, 包括文件描述符、文件名、文件大小、创建时间、当前读写位置等都由操作系统内核来维护, 这些信息也称为文件的元数据 (metadata)。用户程序可以通过 stat() 或 fstat() 函数来查看文件元数据。stat 第一个参数指出的是文件名, 而 fstat 指出的是文件描述符, 这两个函数除了第一个参数类型不同外, 其他方面全部一样。文件的元数据信息用 stat 数据结构描述如下:

```
struct stat {
    dev_t      st_dev;      /* 包含该文件的文件目录项的设备 ID */
    ino_t      st_ino;      /* 节点编号, 在给定文件系统中能唯一标识该文件 */
    mode_t     st_mode;     /* 文件访问权限和文件类型 */
    nlink_t    st_nlink;    /* 连接链的数目 */
```

```

uid_t      st_uid;      /* 文件拥有者的 ID */
gid_t      st_gid;      /* 文件拥有者所在组的组 ID */
dev_t      st_rdev;     /* 设备 ID, 仅对于特殊的块设备和字符设备文件有效 */
off_t      st_size;     /* 普通的磁盘文件的大小, 对于特殊块设备和字符设备文件无效 */
unsigned long st_blksize; /* 块大小 */
unsigned long st_blocks; /* 分配的块数 */
time_t      st_atime;    /* 最近一次访问的时间 */
time_t      st_mtime;    /* 最近一次修改的时间 */
time_t      st_ctime;    /* 最近一次修改文件状态的时间 */
};

```

7. close 系统调用

用法: `close(int fd);`

该函数的功能就是关闭文件 *fd*。

8.2.4 C 标准 I/O 库函数

在 8.2.1 节中提到, 标准 I/O 库函数是基于系统级 I/O 函数实现的。本节通过若干例子介绍如何通过系统级 I/O 函数来实现 C 标准 I/O 库函数。

C 标准 I/O 库函数将一个打开的文件抽象为一个类型为 `FILE` 的“流”模型。上面曾提到, C 标准 I/O 库函数中, 文件是用一个指向特定结构的指针来标识的, 这个特定结构就是 FILE 结构, 它描述了包含文件描述符在内的一组信息。FILE 结构在头文件 `stdio.h` 中描述, 此外, `stdio.h` 文件中还对其他与标准 I/O 有关的常量、数据结构、函数等进行了定义。

以下是从一个典型的 `stdio.h` 文件中摘录的部分内容 (摘自 Brian W. Kernighan, Dennis M. Ritchie, 《The C Programming Language (Second Edition)》)。

```

#define NULL 0
#define EOF (-1)
#define BUFSIZ 1024
#define OPEN_MAX 20 /* max #files open at once */

typedef struct _iobuf{
    int cnt; /* characters left */
    char *ptr; /* next character position */
    char *base; /* location of buffer */
    int flag; /* mode of file access */
    int fd; /* file descriptor */
} FILE;
extern FILE _iob[OPEN_MAX];

#define stdin (&_iob[0])
#define stdout (&_iob[1])
#define stderr (&_iob[2])

enum _flags {
    _READ = 01, /* file open for reading */
    _WRITE = 02, /* file open for writing */
    _UNBUF = 04, /* file is unbuffered */
    _EOF = 010, /* EOF has occurred on this file */
    _ERR = 020 /* error occurred on this file */
};

```

```

int _fillbuf(FILE *);
int _flushbuf(int, FILE *);

#define feof(p) (((p) -> flag & _EOF) != 0)
#define ferror(p) (((p) -> flag & _ERR) != 0)
#define fileno(p) ((p) -> fd)

#define getc(p) (--(p) -> cnt >= 0 ? (unsigned char)*(p) -> ptr++ : _fillbuf(p))
#define putc(x,p) (--(p) -> cnt >= 0 ? *(p) -> ptr++ = (x) : _flushbuf((x),p))

#define getchar() getc(stdin)
#define putchar(x) putc((x), stdout)

```

类型为 FILE 的“流”是对文件描述符和流缓冲区的抽象，流缓冲区由缓冲区起始位置 *base*、下一个可读写位置 *ptr* 以及剩下未读写的字节数 *cnt* 来描述。

在上述 `stdio.h` 文件中，定义了三个特殊的文件，它们分别是标准输入（`stdin`）、标准输出（`stdout`）和标准错误（`stderr`），它们分别定义为打开的文件列表中的第 0~2 个文件，对应的文件描述符分别是 0、1 和 2，它们在结构数组 `_iob` 中前三项的初始化定义如下：

```

FILE _iob[OPEN_MAX] = { /* stdin, stdout, stderr: */
    { 0, (char *) 0, (char *) 0, _READ, 0 },
    { 0, (char *) 0, (char *) 0, _WRITE, 1 },
    { 0, (char *) 0, (char *) 0, _WRITE | _UNBUF, 2 },
};

```

这三个标准文件的流缓冲区的初始化信息相同，其起始位置、下一个可读写位置以及剩下未读写字节数都被初始化为 0。标准输入的访问模式是只读（`_READ`），标准输出和标准错误是可写（`_WRITE`），但前者带有缓冲，而后者无缓冲。

在 `stdio.h` 中定义了 `feof()`、`ferror()`、`fileno()`、`getc()`、`putc()`、`getchar()`、`putchar()` 等标准 I/O 库函数。

从上述 8.2.2 节和 8.2.3 节中的内容可知，系统级 I/O 函数中对文件的标识仅是文件描述符，而 C 标准 I/O 库函数中对文件的标识是指向 FILE 结构的指针，因此，它多了一个流缓冲区。

系统级 I/O 函数的功能是通过执行内核中的系统调用服务例程来实现的，在用户程序中每调用一次系统级 I/O 函数，就是进行一次系统调用。每次系统调用都有两次上下文切换，先从用户态切换到内核态，处理结束后再从内核态返回到用户态，因此，系统调用的开销非常高。例如，在 IA-32 中，因为系统调用属于陷阱类异常，因此，从用户态切换到内核态的过程，正如 7.2.7 节中描述的关于异常和中断响应阶段所做的工作那样，需要进行一系列的操作；而从内核态切换到用户态的过程就是 7.2.7 节中描述的执行 `IRET` 指令的过程。由此可见，每次系统调用会增加许多额外开销，因此，如果能够不用系统调用则应尽量不用，或尽量减少系统调用次数。

在 C 标准 I/O 库函数中引入流缓冲区，可以尽量减少系统调用的次数。因为使用流缓冲区后，可以使用户程序仅和缓冲区进行信息交换，也即，可使文件的内容缓存在用户的缓冲区中，而不是每次都直接读写文件，从而减少执行系统调用的次数。

从 `stdio.h` 中可以看出，有了流缓冲区后，函数 `getc()` 就只要对文件对应流缓冲区的指针进

行修改（如 *cnt* 减 1，*ptr* 加 1）并返回缓冲区中当前所指字符即可。如果流缓冲区的 *cnt* 为负，则说明已经没有字符可读，此时调用函数 `_fillbuf()` 来填充缓冲区。

通常在第一次调用 `getc()` 时，需要调用 `_fillbuf()` 函数进行缓冲区填充。如图 8.6 所示，在 `_fillbuf()` 函数中，若发现文件的打开模式不是 `_READ`（对应 *mode* 为 ‘r’ 的情况）时，就立即返回 `EOF`；否则，它会试图分配一个缓冲区（如果是带缓冲读写的情况）。一旦缓冲区建立后，`_fillbuf()` 就会执行 `read` 系统调用，以读入最多 1024（`BUFSIZ = 1024`）个字节到缓冲区中，并设定下一指针 *ptr* 和剩余字节数 *cnt* 等。图 8.6 中给出了 `_fillbuf()` 函数的源代码（摘自 Brian W. Kernighan, Dennis M. Ritchie, 《The C Programming Language (Second Edition)》）。

```
#include "syscalls.h"

/* _fillbuf: allocate and fill input buffer */
int _fillbuf(FILE *fp)
{
    int bufsize;

    if ((fp->flag & ( _READ | _EOF | _ERR)) != _READ)
        return EOF;
    bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZ;
    if ((fp->base == NULL) /* no buffer yet */)
        if ((fp->base = (char *) malloc(bufsize)) == NULL)
            return EOF; /* can't get buffer */
    fp->ptr = fp->base;
    fp->cnt = read (fp->fd, fp->ptr, bufsize);
    if (--fp->cnt < 0) {
        if (fp->cnt == -1) fp->flag |= _EOF;
        else fp->flag |= _ERR;
        fp->cnt = 0;
        return EOF;
    }
    return (unsigned char) *fp->ptr++;
}
```

图 8.6 分配并填充缓冲区函数 `_fillbuf()` 的实现

假定有一个重复调用 `getc()` 函数 *n* 次的应用程序，第一次调用 `getc()` 时，实际上一下子通过 `read()` 读入了 1024 个字节到一个流缓冲区中，以后每次调用只要从该流缓冲区读取并返回字符即可。这样，若 *n* < 1024，则执行 `read` 系统调用的次数为 1。如果应用程序直接调用函数 `read()` 而不是调用标准 I/O 库函数 `getc()`，那么，应用程序就要执行 *n* 次 `read` 系统调用，从而增加许多额外开销。

例 8.1 假设函数 `filecopy()` 的功能是从一个输入文件复制信息到另一个输出文件，比较以下两种实现方式下系统调用的次数。

```
/* 方式一: getc/putc 版本 */
void filecopy(FILE *infp, FILE *outfp)
{
    int c;
    while ((c = getc(infp)) != EOF)
        putc(c, outfp);
}
```

```

/* 方式二: read/write 版本 */
void filecopy(int *infp, int *outfp)
{
    char c;
    while (read(infp, &c, 1) != 0)
        write(outfp, &c, 1);
}

```

解 显然, 方式二的系统调用次数更多, 因为每次调用 `read()` 和 `write()` 都只进行一个字符的读和写, 因此, 对于文件长度为 n 的情况, 共需执行 $2n$ 次系统调用。方式一用 `getc` 函数读取输入文件中的字符, 第一次读取文件时会通过 `read` 系统调用将最多 1024 个字符一次读入流缓冲区中, 这样, 以后每次读取字符时就直接从流缓冲区读入, 而无需调用 `read` 系统调用, 因而, 若输入文件长度小于 1024 字节, 那么对于读操作只有一次系统调用。■

前面已经提过, C 标准 I/O 库函数是基于底层的系统调用函数实现的, 从上述函数 `_fillbuf()` 的实现也可以看出这一点。

以下用标准 I/O 库函数 `fopen()` 的实现作为例子来说明如何基于底层的系统级 I/O 函数实现 C 标准 I/O 库函数。 `fopen()` 的用法如下:

```

#include <stdio.h>
FILE *fopen(char *name, char *mode);

```

`fopen()` 函数的功能是打开文件名为 `name` 的文件, 具体来说, 主要是分配一个 `FILE` 结构, 并初始化其中的流缓冲区, 返回指向该 `FILE` 结构的指针, 不管由于什么原因不能打开文件, 都返回 `NULL`。

参数 `mode` 指出用户程序将如何使用文件, 可以是 “`rwab+`” 中的一个或多个字符构成的字符串, 例如, ‘`r`’、‘`w`’、‘`a`’、‘`a+b`’ 等。各字符的含义如下:

- 若 `mode` 中有 ‘`a`’ (append), 则数据只能写到文件末尾, 且当前读写位置为文件尾。
- 若 `mode` 中有 ‘`r`’ (read), 则文件必须已经存在并包含了数据, 若不存在或不能被打开, 则返回 `NULL`; 数据只能写到文件末尾, 且当前读写位置为文件尾。
- 若 `mode` 中有 ‘`w`’ (write), 则对于已存在的文件就截断到 0 字节, 对于不存在的文件就创建它。
- 若 `mode` 中有 ‘`+`’ (update), 则允许从该文件读取数据或写入数据。如果和 ‘`r`’ 或 ‘`w`’ 一起使用, 则可以在任意一处从文件读取或写入。如果和 ‘`a`’ 一起使用, 则数据只能写入文件尾部。
- 若 `mode` 中有 ‘`b`’ (binary), 则表示文件按二进制形式打开, 否则按文本形式打开。

假定系统级 I/O 函数定义包含在头文件 `syscalls.h` 中, C 标准 I/O 库函数 `fopen()` 的一个实现版本如图 8.7 所示 (摘自 Brian W. Kernighan, Dennis M. Ritchie, 《The C Programming Language (Second Edition)》)。

图 8.7 给出的实现版本没有对所有访问模式进行处理, 缺少了针对 ‘`b`’ 和 ‘`+`’ 情况的处理。从图 8.7 可以看出, `fopen()` 是通过系统调用 `open` 和 `creat` 来实现的。在用 `fopen()` 打开文件后, 就可以对其进行读写。通常, 第一次读取一个打开的文件时, 会像函数 `_fillbuf()` 实现的那样, 先将文件中的一块数据读出填充到文件对应的流缓冲区, 以后若需要读取这一块数据中

的信息时，就可以从对应流缓冲区读取。

```
#include <fcntl.h>
#include "syscalls.h"
#define PERMS 0666 /* RW for owner, group, others */

/* fopen: open files, return file ptr */
FILE *fopen(char *name, char *mode)
{
    int fd;
    FILE *fp;

    if (*mode != 'r' && *mode != 'w' && *mode != 'a')
        return NULL;
    for (fp = _iob; fp < _iob + OPEN_MAX; fp++)
        if ((fp->flag & ( _READ | _WRITE )) == 0)
            break; /* found free slot */
    if (fp >= _iob + OPEN_MAX) /* no free slots */
        return NULL;

    if (*mode == 'w') fd = creat(name, PERMS);
    else if (*mode == 'a') {
        if ((fd = open(name, O_WRONLY, 0)) == -1)
            fd = creat(name, PERMS);
        lseek(fd, 0L, 2);
    } else fd = open(name, O_RDONLY, 0);
    if (fd == -1) return NULL; /* couldn't access name */
    fp->fd = fd;
    fp->cnt = 0;
    fp->base = NULL;
    fp->flag = (*mode == 'r') ? _READ : _WRITE;
    return fp;
}
```

图 8.7 标准 I/O 库函数 fopen() 的一种实现版本

8.2.5 用户程序中的 I/O 请求

在用户空间 I/O 软件中，用户程序可以使用 C 标准 I/O 库函数的方式提出 I/O 请求，也可以直接使用操作系统提供的系统级 I/O 函数或 API 函数提出 I/O 请求。

例如，对于一个简单的文件复制功能，可以使用多种不同的实现方式。例 8.1 中给出了两种实现方式，而如图 8.8 所示的是另一种方式，它使用 C 标准 I/O 库函数 fread() 和 fwrite() 来实现。

对于文件复制功能，还可用函数 fgetc() 和 fputc() 来实现，其程序类似图 8.8 所示的程序，所不同的只是将第 25 行到第 31 行替换为如下语句：

```
while (! feof (srcfile)) fputc(fgetc(srcfile), dstfile);
```

在 Windows 系统中，除了使用 C 标准库函数实现以外，还可使用 API 函数 ReadFile() 和 WriteFile() 来实现文件复制功能。此外，操作系统还可能会提供一些更高级的 API 函数，它通过组合若干基本 API 函数而形成，是用于完成特定功能的、抽象度更高的 API 函数。例如，在

Windows 系统中，提供了一个特定的用于文件复制的函数 CopyFile()，它通过调用基本 API 函数 CreateFile()、ReadFile()、WriteFile() 和 CloseHandle() 来实现，用户程序可以直接使用 CopyFile() 函数来实现文件复制功能。

```
1  #include <stdio.h>
2  #include <errno.h>
3
4  int main ( int argc, char *argv[] )
5  {
6      FILE *srcfile, *dstfile;
7      char buf[BUFSIZ];
8      size_t srcsize, dstsize;
9
10     if ( argc != 3 ) {
11         printf( "usage: copyfile srcfile dstfile\n" );
12         return 1;
13     }
14     srcfile=fopen(argv[1], "rb");
15     if ( srcfile == NULL ) {
16         perror(argv[1]);
17         return 2;
18     }
19     dstfile=fopen(argv[2], "wb");
20     if ( dstfile == NULL ) {
21         perror(argv[2]);
22         return 3;
23     }
24
25     while ((srcsize=fread(buf, 1, BUFSIZ, srcfile)) > 0) {
26         dstsize=fwrite(buf, 1, srcsize, dstfile);
27         if (dstsize != srcsize) {
28             perror("Write Error.");
29             return 4;
30         }
31     }
32     fclose(srcfile);
33     fclose(dstfile);
34     return 0;
35 }
```

图 8.8 使用 C 标准 I/O 库函数示例程序

上面就文件复制功能列举了多种不同的实现方式，不管用户空间 I/O 软件通过何种方式来提出 I/O 请求，编译器和汇编器最终都会将 I/O 请求转换为如图 8.4 中第 3~7 行所示的若干条指令。其中，传送指令用来准备系统调用的调用号和参数，通常，调用号被传送到 EAX 寄存器，而参数被传送到其他通用寄存器。例如，在 IA-32/Linux 系统中，参数将依次被存放在 EBX、ECX、EDX 等通用寄存器中。也可以不将参数送到通用寄存器，而是将参数在栈中的起始地址送到特定的通用寄存器，这种做法在参数较多时比较合适。例如，在 IA-32/Windows 系统中就是这样传递参数的，通常将第一个参数在栈中的地址送到 EDI 寄存器，因为调用系统调用的封装函数时，参数已经按序压入栈中，所以，通过 EDI 可以访问到栈中所有参数。在传送指令后面是一条用于系统调用的陷阱指令，在 IA-32/Linux 系统中，通常是“int \$0x80”，

在 IA-32/Windows 系统中通常是“int \$0x2e”指令。这两种系统也都可以用快速系统调用指令 sysenter 进行系统调用。

8.3 I/O 硬件与软件的接口

用户空间 I/O 软件中的 I/O 请求最终是通过一条陷阱指令转入内核由内核空间中的 I/O 软件来控制 I/O 硬件完成的。因为内核空间中底层 I/O 软件的编写与 I/O 硬件的结构密切相关，所以在介绍内核空间的 I/O 软件之前，先介绍 I/O 硬件的基本组成。对于编写内核空间 I/O 软件的程序员来说，所关心的是 I/O 硬件中与软件的接口部分，因此，本节主要介绍与软件相关的 I/O 硬件部分，而不是介绍如何设计和制造 I/O 硬件的物理部件。

I/O 硬件通常由机械部分和电子部分组成，并且两部分通常是可以分开的。机械部分是 I/O 设备本身，而电子部分则称为设备控制器或I/O 适配器。

8.3.1 I/O 设备

I/O 设备又称外围设备、外部设备，简称外设，是计算机系统与人类或其他计算机系统之间进行信息交换的装置。操作系统为了更好地对 I/O 设备进行统一管理，通常将 I/O 设备分成两类：字符设备和块设备。

字符设备是以字符为单位向主机发送或从主机接收一个字符流的设备。字符设备传送的字符流不能形成数据块，无法对其进行定位和寻址。

通常，大多数输入设备和输出设备都可以看作是一种字符设备。输入设备的功能是把数据、命令、字符、图形、图像、声音或电流、电压等信息，以计算机可以接受或识别的二进制代码形式输入到计算机中，例如，键盘、鼠标、触摸屏、跟踪球、控制杆、数字化仪、扫描仪、手写笔、光学字符阅读机等都是输入设备；输出设备的功能是把计算机处理的结果，变成最终可以被人理解的数据、文字、图形、图像和声音等信息，例如，显示器、打印机和绘图仪等都是输出设备。

还有一类主要用于计算机和计算机之间通信的设备，称为机-机通信设备，例如，网络接口、调制解调器、数/模和模/数转换器等。通常，大多数机-机通信设备也可看作是一种字符设备。

块设备以一个固定大小的数据块为单位与主机交换信息，通常，外部存储器是块设备，例如，磁盘驱动器、光盘驱动器和磁带机等。块设备中的数据块的大小通常在 512 字节以上，它按照某种组织方式被写入或读出设备，每个数据块都有唯一的位置信息，因而是可寻址的。典型的块设备是磁盘，有关磁盘存储器的结构、性能指标及其在计算机系统的位置和连接等内容请参看 6.3 节。

操作系统将所有设备划分成字符设备和块设备两类，主要是为了便于操作系统对各种不同字符设备和不同块设备的共同特点进行抽象，从而在实现操作系统中的 I/O 软件时，可以尽可能多地划分出与设备无关的软件部分。例如，对于块设备，可以在文件系统只处理与设备无关的抽象块设备，而把与设备相关的部分放到更低层次的设备驱动程序中实现。

8.3.2 设备控制器

图 8.9 给出了外设通过设备控制器和 I/O 总线与 CPU 及主存储器相互连接的示意图。

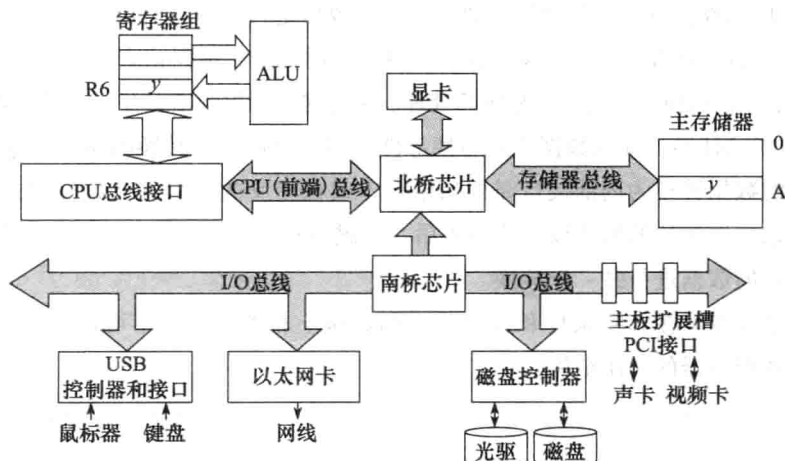


图 8.9 外设、设备控制器和 CPU 及主存的连接

设备控制器又称 I/O 控制器 或 I/O 控制接口，简称 I/O 模块 或 I/O 接口，是介于外设和 I/O 总线之间的部分，不同的外设往往对应不同的设备控制器。设备控制器通常独立于 I/O 设备，可以集成在主板上或以插卡的形式插在扩展槽上。例如，图 8.9 中的 磁盘控制器、以太网卡（网络控制器）、USB 控制器 等都是一种设备控制器。它根据从 CPU 接收到的控制命令来对相应外设进行控制。它在主机一侧与 I/O 总线相连，在外设一侧提供相应的 连接器插座，在插座上连上相应的连接外设的电缆，就可以将外设通过设备控制器连接到主机。

图 8.10 给出了常用的几种连接外设的插座。目前很多外设都可以连接在 USB 接口上，例如，键盘和鼠标可以连接在 PS/2 插座（图 8.10 中的键盘接口和鼠标器接口处的插座）上，也可以连在 USB 接口上。

I/O 总线 用来在系统中连接多个设备控制器和总线桥接器，物理上它是主板上的一些 I/O 扩展槽，通常是标准总线。目前典型的 I/O 总线标准是 PCI-Express，早期的 I/O 总线标准有 ISA、EISA、PCI、AGP、PCI-X 等。

设备控制器的主要职能包括以下几个方面。

① 数据缓冲。主存和 CPU 寄存器的存取速度都非常快，而外设的速度则较低，在设备控制器中引入 数据缓冲寄存器 后，输出数据时，CPU 只要把数据送到数据缓冲寄存器即可；在输入数据时，CPU 只要从数据缓冲寄存器取数即可。在设备控制器控制将数据缓冲寄存器的数据输出到外设或从外设读入数据时，CPU 可以做

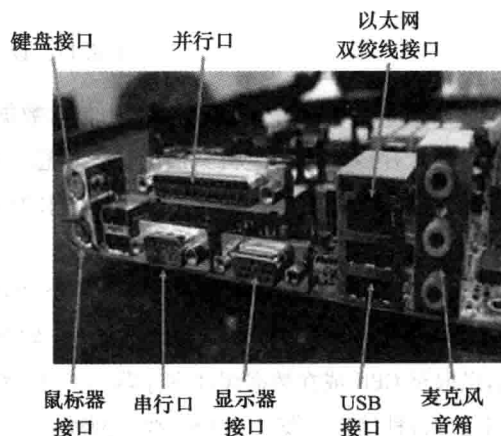


图 8.10 常用 I/O 设备插座

其他事情。

② 错误和就绪检测。提供了错误和就绪检测逻辑，并将结果保存在状态寄存器，以供 CPU 查用。状态信息包括各类就绪和错误信息，如：外设是否完成打印或显示、是否准备好输入数据以供 CPU 读取、打印机是否发生缺纸、磁盘数据是否发生检验错等。

③ 控制和定时。接收主机侧送来的控制信息和定时信号，根据相应的定时和控制逻辑，向外发送控制信号，以控制外设进行相应的处理。主机送来的控制信息存放在控制寄存器中。

④ 数据格式的转换。提供数据格式转换部件（如进行串 - 并转换的移位寄存器），使通过外部接口得到的数据转换为内部接口需要的格式，或在相反的方向进行数据格式转换。例如，从磁盘驱动器以二进制位的形式读出或写入后，在磁盘控制器中，应对读出的数据进行串 - 并转换，或对写入的数据进行并 - 串转换。

不同的设备控制器在其复杂性和控制外设的数量上相差很大，不可能一一列举。图 8.11 给出了一个设备控制器的通用结构。

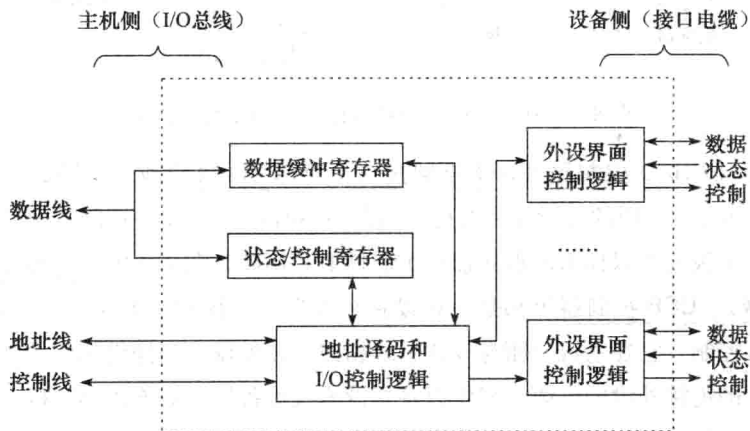


图 8.11 设备控制器的通用结构

如图 8.11 所示，设备控制器中包含数据缓冲寄存器、状态/控制寄存器等多个不同的寄存器，用于存放外设与主机交换的数据信息、控制信息和状态信息。因为状态信息和控制信息在传送方向上是相反的，而且 CPU 对它们的访问在时间上也是错开的，所以，在有些设备控制器中将它们合二为一。

设备控制器是连接外设和主机的一个“桥梁”，它在外设侧和主机侧各有一个接口。设备控制器在主机侧通过 I/O 总线和主机相连，实现将控制信息送控制寄存器、将状态寄存器中的状态信息取至 CPU 或在数据缓冲寄存器与 CPU 寄存器之间进行数据交换的功能；设备控制器在外设侧通过各种接口电缆（如 USB 线、网线、并行电缆等）和外设相连。因此，通过连接电缆、设备控制器、各类总线及其桥接器，可以在 I/O 硬件、主存和 CPU 之间建立一个信息传输“通路”。

有了设备控制器，底层 I/O 软件就可以通过设备控制器来控制外设，因而编写底层 I/O 软件的程序员只需要了解设备控制器的工作原理，包括：设备控制器中有哪些用户可访问的寄存器、控制/状态寄存器中每一位的含义、设备控制器与外设之间的通信协议等，而关于外设的机械特性，程序员则无需了解。

在底层 I/O 软件中, 可以将控制命令送到控制寄存器来启动外设工作; 可以读取状态寄存器来了解外设和设备控制器的状态; 可以通过直接访问数据缓冲寄存器来进行数据的输入和输出。当然, 这些对数据缓冲寄存器、控制/状态寄存器的访问操作是通过相应的指令来完成的, 通常把这类指令称为 I/O 指令。因为这些 I/O 指令只能在操作系统内核的底层 I/O 软件中使用, 因而它们是一种特权指令。

例如, 在 IA-32 中, 提供了 4 条专门的 I/O 指令: in、ins、out 和 outs。其中 in 和 ins 指令用于将设备控制器中某个寄存器的内容取到 CPU 内的通用寄存器中; out 和 outs 用于将通用寄存器的内容输出到设备控制器的某个寄存器中。

8.3.3 I/O 端口及其编址

通常把设备控制器中的数据缓冲寄存器、状态/控制寄存器等统称为 I/O 端口 (I/O port)。数据缓冲寄存器简称为 数据端口, 状态/控制寄存器简称为 状态/控制端口。为了便于 CPU 对外设的快速选择和对 I/O 端口的寻址, 必须对 I/O 端口进行编址, 所有 I/O 端口编号组成的空间称为 I/O 地址空间。I/O 端口的编址方式有两种: 统一编址方式和独立编址方式。

1. 统一编址方式

统一编址方式下, I/O 地址空间与主存地址空间统一编址, 也即, 将主存地址空间分出一部分地址给 I/O 端口进行编号, 因为 I/O 端口和主存单元在同一个地址空间的不同分段中, 因而根据地址范围就可区分访问的是 I/O 端口还是主存单元, 因此也就无需设置专门的 I/O 指令, 只要用一般的访存指令就可以存取 I/O 端口。因为这种方法是将 I/O 端口映射到主存空间的某个地址段上, 所以也被称为“存储器映射方式”。

因为统一编址方式下 I/O 访问和主存访问共用同一组指令, 所以它的保护机制可由分段或分页存储管理来实现, 而无需专门的保护机制。这种存储器映射方式给编程提供了非常大的灵活性。任何对内存存取的指令都可用来访问位于主存空间中的 I/O 端口, 并且所有有关主存的寻址方式都可用于 I/O 端口的寻址。例如, 可用访存指令实现 CPU 中通用寄存器和 I/O 端口的数据传送; 可用 AND、OR 或 TEST 等指令直接操作设备控制器中的控制寄存器或状态寄存器。

采用统一编址的另一个好处是便于扩大系统吞吐率, 因为外设或 I/O 端口数目除了受总存储容量的限制外, 几乎不受其他因素的限制。这在大型控制或数据通信系统等特殊场合很有用。不过, 因为 I/O 空间占用了一部分主存空间的地址, 因而会减少可寻址的主存空间。此外, 由于在识别 I/O 端口时全部地址线都需参与地址译码, 使译码电路变复杂了, 并需用较长时间进行地址译码, 所以寻址时间变长了。

2. 独立编址方式

独立编址方式对所有的 I/O 端口单独进行编号, 使它们成为一个独立的 I/O 地址空间。这种情况下, 指令系统中需要有专门的 I/O 指令来访问 I/O 端口, I/O 指令中地址码部分给出 I/O 端口号。

独立编址方式中, I/O 地址空间和主存地址空间是两个独立的地址空间, 因而无法从地址码的形式上区分 CPU 访问的是 I/O 端口还是主存单元, 故需用专门的 I/O 指令来表明访问的是 I/O 地址空间。CPU 执行 I/O 指令时, 会产生 I/O 读或 I/O 写总线事务, CPU 通过 I/O 读或

I/O 写总线事务访问 I/O 端口。

通常, I/O 端口数比存储器单元少得多, 选择 I/O 端口时, 只需少量地址线, 因此, 在设备控制器中的地址译码逻辑比较简单, 寻址速度快。独立编址的另一好处是, 因为使用专用 I/O 指令, 使得程序的结构比较清晰, 很容易判断出哪部分代码是用于 I/O 操作的, 因而便于理解代码以及检查代码的正确性, 不过, I/O 指令往往只提供简单的传输操作, 故程序设计的灵活性差一些。

IA-32 采用独立编址方式, 有专门的 I/O 指令: in (ins) 和 out (outs)。在 IA-32 中, I/O 地址空间中共有 65 536 个 8 位的 I/O 端口, 可以把两个连续的 8 位端口看成一个 16 位端口。

8.3.4 I/O 控制方式

通过连接电缆、设备控制器和各类总线及其桥接器, 在 CPU、主存和 I/O 硬件之间建立了一个信息传输“通路”。底层 I/O 软件利用这个“通路”, 通过读写设备控制器中的各类寄存器来控制设备进行输入输出。

I/O 操作可以有三种不同控制方式: 程序直接控制、中断控制和 DMA 控制。

1. 程序直接控制 I/O 方式

程序直接控制 I/O 方式的基本实现思想是, 直接通过查询程序来控制主机和外设的数据交换, 因此, 也称为程序查询或轮询 (polling) 方式。该方式在查询程序中安排相应的 I/O 指令, 通过这些指令直接向设备控制器传送控制命令, 并从状态寄存器中取得外设和设备控制器的状态后, 根据状态来控制外设和主机的数据交换。

下面以打印输出一个字符串为例来说明其基本原理。假定一个用户程序 P 中使用了某个 I/O 函数, 请求在打印机上打印一个具有 n 个字符的字符串。显然, 用户进程 P 通过一系列过程调用后, 会执行一个系统调用来打开打印机设备。若打印机空闲, 则用户进程可正常使用打印机, 因而用户进程就通过另一个系统调用来对打印机进行写操作, 从而陷入操作系统内核进行字符串打印。

操作系统内核通常将用户进程缓冲区中的字符串首先复制到内核空间, 然后查看打印机是否“就绪”。如果“就绪”, 则将内核空间缓冲区中的一个字符输出到打印控制器的数据端口中, 并发出“启动打印”命令, 以控制打印机打印数据端口中的字符; 如果打印机“未就绪”, 则等待直到其“就绪”。上述过程循环执行, 直到字符串中所有字符打印结束。图 8.12 中的程序段大致描述了上述过程。

```
copy_string_to_kernel(strbuf, kernelbuf, n);           // 将字符串复制到内核缓冲区
for (i=0; i < n; i++) {                                // 对于每个打印字符循环执行
    while (printer_status != READY);                    // 等待直到打印机状态为“就绪”
    *printer_data_port=kernelbuf[i];                    // 向数据端口输出一个字符
    *printer_control_port=START;                         // 发送“启动打印”命令
}
return();                                              // 返回
```

图 8.12 程序直接控制 I/O 的一个例子

打印机的“就绪”和“缺纸”等状态记录在打印控制器的状态端口中。“启动打印”命令被送到打印控制器的控制端口。打印控制器在每次打印完当前数据端口中的字符时，自动将“就绪”状态置1，以表明打印控制器中的数据端口已准备就绪，CPU 可以向数据端口送入新的欲打印字符。

程序直接控制 I/O 方式的特点是简单、易控制、设备控制器中的控制电路简单。但是，CPU 需要从设备控制器中读取状态信息，并在外设未就绪时一直处于忙等待。因为外设的速度比 CPU 慢得多，因此，在 CPU 等待外设完成任务的过程中浪费了大量的处理器时间。

2. 中断控制 I/O 方式

中断控制 I/O 方式的基本思想是，当需要进行 I/O 操作时，首先启动外设进行第一个数据的 I/O 操作，然后使 CPU 转去执行其他用户进程，而请求 I/O 的用户进程被阻塞。在 CPU 执行其他进程的过程中，外设在对应设备控制器的控制下进行数据的 I/O 操作。当外设完成 I/O 操作后，向 CPU 发送一个中断请求信号，CPU 检测到有中断请求信号后，就暂停正在执行的进程，并调出相应的中断服务程序执行。CPU 在中断服务程序中，再启动随后数据的 I/O 操作，然后中断返回，回到被打断的进程继续执行。

例如，对于上述请求打印字符串的用户进程 P 的例子，如果采用中断控制 I/O 方式，则操作系统处理 I/O 的过程如图 8.13 所示。

```
copy_string_to_kernel(strbuf, kernelbuf, n);           // 将字符串复制到内核缓冲区
enable_interrupts();                                   // 开中断，允许外设发出中断请求
while (printer_status != READY);                       // 等待直到打印机状态为“就绪”
*printer_data_port=kernelbuf[i];                     // 向数据端口输出第一个字符
*printer_control_port=START;                          // 发送“启动打印”命令
scheduler();                                           // 阻塞用户进程P，调度其他进程执行
```

a) “字符打印”系统调用服务例程

```
if (n==0) {                                           // 若字符串打印完，则
    unblock_user();                                  // 用户进程P解除阻塞，P进入就绪队列
} else {
    *printer_data_port=kernelbuf[i];                // 向数据端口输出一个字符
    *printer_control_port=START;                    // 发送“启动打印”命令
    n = n-1;                                         // 未打印字符数减1
    i = i+1;                                         // 下一个打印字符指针加1
}
acknowledge_interrupt();                             // 中断回答（清除中断请求）
return_from_interrupt();                             // 中断返回
```

b) “字符打印”中断服务程序

图 8.13 中断控制 I/O 的一个例子

从图 8.13a 可以看出，在“字符打印”系统调用服务例程中启动打印机后，它就调用处理器调度程序 scheduler 来调出其他进程执行，而将用户进程 P 阻塞。在 CPU 执行其他进程的同时，打印机在进行打印操作，CPU 和打印机并行工作。若打印机打印一个字符需要 5ms，则在打印一个字符期间，其他进程可以在 CPU 上执行 5ms 的时间。对于程序直接控制 I/O 方式，

CPU 在这 5ms 的时间内只是不断地查询打印机状态，因而整个系统的效率很低。

小贴士

在多道程序（多任务）系统中，单个处理器可以被多个进程共享，即多个进程可以轮流使用处理器。为此，操作系统必须使用某种调度方法决定何时停止一个进程在处理器上的运行，转而使处理器运行另一个进程。操作系统中使用某种调度方法进行处理器调度的程序称为处理器调度程序。

简单来说，一个进程有三种状态：运行、就绪和阻塞。正在处理器上运行着的进程处于运行态；可以被调度到处理器运行但因为时间片到等原因被换下的进程处于就绪态；因为某种事件的发生而不能继续在处理器上运行的进程处于阻塞态。处于阻塞态的进程也称为被挂起，典型的处于阻塞态进程的例子就是等待 I/O 完成的进程，因为 I/O 操作没有完成的话，进程便无法继续运行下去。处于就绪态的进程可能有多个，为方便选择就绪态进程运行，通常将所有就绪态进程组成一个就绪队列，解除阻塞的进程可进入就绪队列。

中断控制 I/O 方式下，一旦外设完成任务，就会向 CPU 发中断请求。对于图 8.13 中的例子，当打印机完成一个字符的打印后，就会发中断请求，然后 CPU 暂停正在执行的其他进程，调出“字符打印”中断服务程序来执行。如图 8.13b 所示，中断服务程序首先判断是否已完成字符串中所有字符的打印，若是，则将用户进程 P 解除阻塞，使其进入就绪队列；否则，就向数据端口送出下一个欲打印字符，并启动打印，将未打印字符数减 1 和下一个打印字符指针加 1 后，执行中断返回，回到被打断的进程继续执行。

图 8.14 和图 8.15 描述了中断控制 I/O 的整个过程。

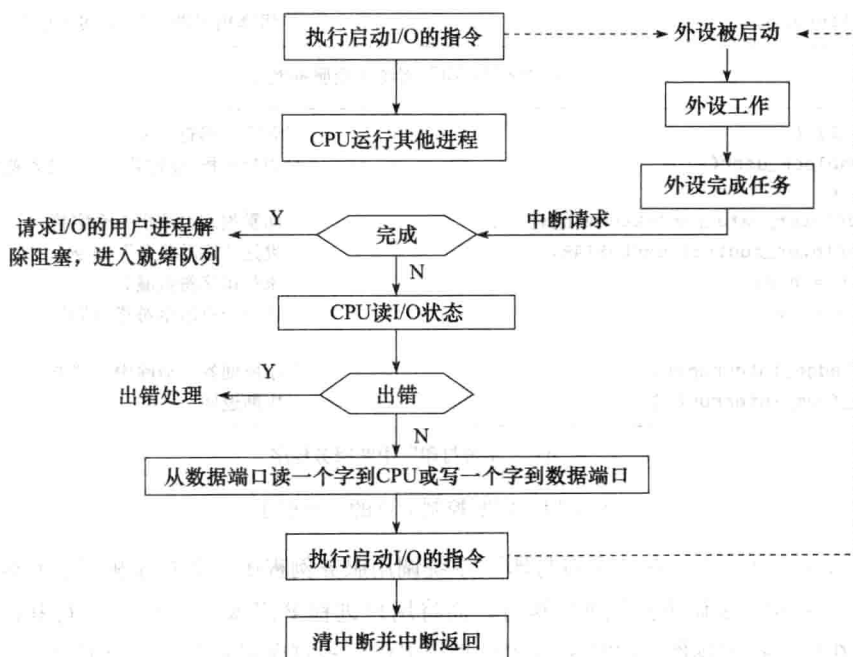


图 8.14 中断控制 I/O 过程

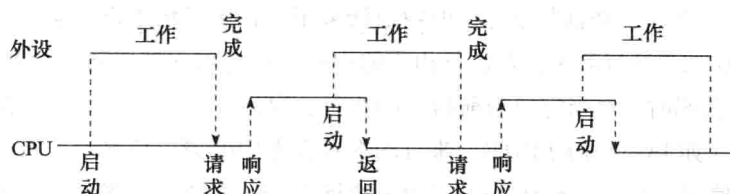


图 8.15 CPU 与外设并行工作

现代计算机系统的中断处理功能相当丰富，没有配置中断系统的计算机是令人无法想象的。每个计算机系统的中断功能可能不完全相同，但中断系统的基本功能无外乎以下几个方面：① 及时记录各种中断请求信号，通常用一个中断请求寄存器来保存；② 自动响应中断请求，CPU 在每条指令执行完后，会自动检测中断请求引脚，发现有中断请求后会自动响应中断；③ 在同时有多个中断请求时，能自动选择响应优先级最高的中断请求；④ 保护被中断程序的断点和现场，断点指中断返回时返回到被中断程序继续执行时那条指令的地址，而现场则是指被中断程序所使用的通用寄存器的内容，断点由 CPU 保存，现场由中断服务程序保存；⑤ 通过中断屏蔽实现多重中断的嵌套执行，中断屏蔽功能通常用一个中断屏蔽字寄存器来实现。

现代计算机系统大多采用中断嵌套技术。也就是说，中断系统允许 CPU 在执行某个中断服务程序时，被新的中断请求打断。但是并不是所有的中断处理都可被新的中断打断，对于一些重要的紧急事件的处理，就要设置成不可被其他新中断事件打断，这就是中断屏蔽的概念。中断系统中要有中断屏蔽机制，使得每个中断可以设置它允许被哪些中断打断，不允许被哪些中断打断。这个功能主要通过在中断系统中设置中断屏蔽字来实现。屏蔽字中的每一位对应某一个外设中断源，称为该中断源的中断屏蔽位，例如，用“1”表示允许中断，“0”表示不允许中断（即屏蔽中断）。CPU 还可以通过在程序中执行相应的 I/O 指令来修改屏蔽字的内容，从而动态地改变中断处理的先后次序。

中断系统的基本结构如图 8.16 所示。

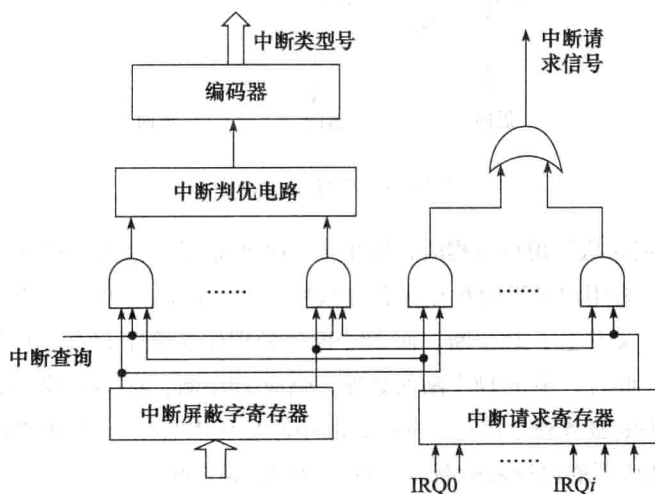


图 8.16 中断系统的基本结构

从图 8.16 可以看出，来自各个外设的中断请求记录在中断请求寄存器中的对应位，每个中断源有各自对应的中断屏蔽字，在进行相应的中断处理之前它被送到中断屏蔽字寄存器中。在 CPU 执行程序代码时，每当完成当前指令的执行、取出下一条指令之前，都会通过采样中断请求信号引脚（如 IA-32 中的 INTR）来自动查看有无中断请求信号。若有，则会发出一个相应的中断回答信号，启动图 8.16 中的“中断查询”线，在该信号线的作用下，所有未被屏蔽的中断请求信号一起送到一个中断判优电路中，判优电路根据中断响应优先级选择一个优先级最高的中断源，然后用一个编码器对该中断源进行编码，得到对应的中断源设备类型号（即中断源的标识信息，称为中断类型号）。CPU 取得中断源的标识信息后，经过一系列相应的转换，就可得到对应的中断服务程序的首地址，在下一个指令周期开始，CPU 执行相应的中断服务程序。

IA-32 系统中，中断系统的功能通过可编程中断控制器（Programmable Interrupt Controller, PIC）实现。如图 8.16 所示，每个能够发出中断请求的外部设备控制器都有一条IRQ 线，所有外设的 IRQ 线连到 PIC 对应的IRQ 引脚：IRQ0、IRQ1、…、IRQ_i，然后被记录在 PIC 的中断请求寄存器中。PIC 对所有外设发来的 IRQ 请求按照优先级进行排队，如果至少有一个 IRQ 线有请求且未被屏蔽，则 PIC 向 CPU 的 INTR 引脚发中断请求。

在中断处理（即执行中断服务程序）过程中，若又有新的处理优先级更高的中断请求发生，那么 CPU 应立即中止正在执行的中断服务程序，转去处理新的中断，这种情况被称为多重中断或中断嵌套，如图 8.17 所示。

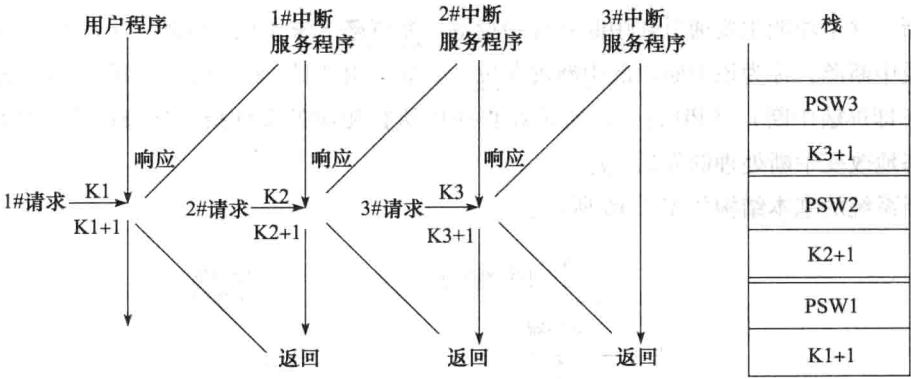


图 8.17 中断嵌套过程

图 8.17 中，假定在执行用户进程时，发生了 1#中断请求，因为用户进程不屏蔽任何中断，所以就响应 1#中断，将用户进程的断点保存在栈中，然后调出 1#中断服务程序执行，而在处理 1#中断的过程中，又发生了 2#中断，而 2#中断的处理优先级比 1#高，也即 1#中断的屏蔽字对 2#中断是开放的，此时，中止 1#中断的处理，响应 2#中断，把 1#中断的断点信息保存在栈中，调出 2#中断的中断服务程序执行。同样，3#中断也可以打断 2#中断的执行。当 3#中断处理完返回时，系统从栈顶取出返回的断点信息，这样，从 3#中断返回后，首先回到 2#中断的断点（K3 + 1）处，而不是回到 1#中断或用户进程执行。利用栈能正确地实现中断嵌套。

从上面描述的过程来看，中断系统中存在两种中断优先级。一种是中断响应优先级，另一

种是中断处理优先级。中断响应优先级是由中断查询程序或硬件判优电路（如图 8.16 中的中断判优电路）决定的优先权，它反映的是多个中断同时请求时选择哪个先被响应；而中断处理优先级是由各自的中断屏蔽字来动态设定的，反映了本中断与其他所有中断之间的处理优先关系。在多重中断系统中通常用中断屏蔽字对中断处理优先权进行动态分配。

中断控制 I/O 方式下，每次执行中断服务程序仅处理一个数据的传送。例如，对于上述字符串打印的例子，每次中断都只打印一个字符，并且，为了响应中断请求和执行中断服务程序，CPU 多执行了许多额外操作，包括保存断点、保存现场、开/关中断、设置中断屏蔽字等各种操作。对于像磁盘这种高速设备的 I/O，如果采用中断控制方式，那么，由于外设数据传输速度快，而 CPU 响应和处理中断的额外开销大，使得 CPU 用于设备 I/O 的时间百分比过大，影响整个系统的效率。

下面的例子说明中断控制 I/O 方式下，CPU 用于磁盘 I/O 的开销。

例 8.2 假定一个字长为 32 位的 CPU 的主频为 500MHz，即 CPU 每秒钟产生 500×10^6 个时钟周期。硬盘使用中断控制 I/O 方式进行数据传送，其传输速率为 4MB/s，每次中断传输一个 16 字节的数据，要求没有任何数据传输被错过。每次中断的开销（包括用于中断响应和中断处理的时间）是 500 个时钟周期。如果硬盘仅有 5% 的时间进行数据传送，那么，CPU 用于硬盘数据传送的时间占整个 CPU 时间的百分比为多少？

解 硬盘数据传送采用中断控制 I/O 方式，每次中断传输一个 16 字节的数据。为保证没有任何数据传输被错过，CPU 每秒钟应该至少执行 $4\text{MB}/16\text{B} = 250 \times 10^3$ 次中断，每秒钟内用于硬盘数据传输的时钟周期数为 $250 \times 10^3 \times 500 = 125 \times 10^6$ ，故 CPU 用于硬盘数据传送的时间占整个 CPU 时间的 $125 \times 10^6 / (500 \times 10^6) = 25\%$ 。

从另外一个角度来考虑也可以得出同样的结论。由题意知，CPU 通过中断控制方式进行硬盘数据的传送，硬盘每准备好一个 16 字节的数据，则发出中断请求。硬盘准备一个数据（16B）的时间为 $16\text{B}/4\text{MB} = 4\mu\text{s}$ ，CPU 用于一个数据输入/输出的时间（包括中断响应并处理的时间）是 500 个时钟周期，相当于 $500/500\text{M} = 1\mu\text{s}$ 。由此可见，假定硬盘一直在工作的话，则硬盘每隔 $4\mu\text{s}$ 申请一次中断，每次中断 CPU 花 $1\mu\text{s}$ 的时间进行硬盘数据传送，因此，CPU 用于硬盘数据传送的时间占整个 CPU 时间的 $1/4$ ，即 25%。

假定硬盘并不是一直在操作，而是仅有 5% 的时间在工作，则 CPU 用于硬盘数据传送的时间占整个 CPU 时间的百分比为 $25\% \times 5\% = 1.25\%$ 。 ■

对于程序直接控制 I/O 方式，在外设准备数据时，由于 CPU 一直在等待外设完成，所以，在这个阶段 CPU 用于 I/O 的时间为 100%。对于中断控制 I/O 方式，在外设准备数据时，CPU 被安排执行其他程序，外设和 CPU 并行工作，因而 CPU 在外设准备数据时没有开销，只有响应和处理中断而进行数据传送时 CPU 才需要花费时间为 I/O 服务。这就是中断控制 I/O 方式相对于程序直接控制 I/O 方式的优点。

但是，对于像硬盘这种高速外设的数据传送，如果还是用中断控制 I/O 方式的话，则 CPU 用于 I/O 的开销是无法忽视的。高速外设速度快，因而中断请求频率高，导致 CPU 被频繁地被打断，而且，由于需要保存断点和现场、开中断/关中断、设置中断屏蔽字等，使得中断响应和中断处理的额外开销很大，因此，在高速外设情况下，采用中断控制 I/O 方式传送数据是

不合适的,通常采用DMA 控制 I/O 方式。

3. DMA 控制 I/O 方式

DMA (Direct Memory Access, 直接存储器访问) 控制 I/O 方式用专门的DMA 接口硬件来控制外设和主存之间的直接数据交换,数据不通过 CPU。通常把专门用来控制数据在主存和外设之间直接传送的接口硬件称为DMA 控制器。

DMA 控制器与设备控制器一样,其中也有若干个寄存器,包括主存地址寄存器、设备地址寄存器、字计数器、控制寄存器等,还有其他的控制逻辑,它能控制设备通过总线与主存直接交换数据。在 DMA 传送前,应先对 DMA 控制器初始化,将需要传送的数据个数、数据所在设备地址以及主存首地址、数据传送的方向(从主存到外设,还是从外设到主存)等参数送到 DMA 控制器。

DMA 控制 I/O 方式的基本思想是,首先对 DMA 控制器进行初始化,然后发送“启动 DMA 传送”命令以启动外设进行 I/O 操作,发送完“启动 DMA 传送”命令后, CPU 转去执行其他进程,而请求 I/O 的用户进程被阻塞。在 CPU 执行其他进程的过程中, DMA 控制器控制外设和主存进行数据交换。DMA 控制器每完成一个数据的传送,就将字计数器减 1,并修改主存地址,当字计数器为 0 时,完成所有 I/O 操作,此时, DMA 控制器向 CPU 发送一个“DMA 完成”中断请求信号, CPU 检测到有中断请求信号后,就暂停正在执行的进程,并调出相应的中断服务程序执行。CPU 在中断服务程序中,解除用户进程的阻塞状态而使用户进程进入就绪队列,然后中断返回,再回到被打断的进程继续执行。

DMA 控制 I/O 方式的处理过程如图 8.18 所示。

copy_string_to_kernel(strbuf, kernelbuf, n);	// 将字符串复制到内核缓冲区
initialize_DMA();	// 初始化DMA控制器 (准备传送参数)
*DMA_control_port=START;	// 发送“启动DMA传送”命令
scheduler();	// 阻塞用户进程, 调度其他进程执行

a) “字符打印”系统调用处理例程

acknowledge_interrupt();	// 中断回答 (清除中断请求)
unblock_user();	// 用户进程P解除阻塞, 进入就绪队列
return_from_interrupt();	// 中断返回

b) “DMA结束”中断服务程序

图 8.18 DMA 控制 I/O 过程

DMA 控制 I/O 方式下, CPU 只要在最初的 DMA 控制器初始化和最后处理“DMA 结束”中断时介入,而在整个一块数据传送过程中都不需要 CPU 参与,因而 CPU 用于 I/O 的开销非常小。

例 8.3 假定 CPU 的主频为 500MHz。硬盘采用 DMA 方式进行数据传送,其数据传输率为 4MB/s,每次 DMA 传输的数据量为 8KB,要求没有任何数据传输被错过。如果 CPU 在 DMA 初始化设置和启动硬盘操作等方面用了 1000 个时钟周期,并且在 DMA 传送完成后的中断处理需要 500 个时钟,则在硬盘 100% 处于工作状态的情况下, CPU 用于硬盘 I/O 操作的时间百分比大约是多少?

解 从硬盘上读/写 8KB 的数据所需时间为 $8\text{KB}/4\text{MB/s} = 2.048\text{ms} \approx 2\text{ms}$ ，如果硬盘一直处于工作状态的话，为了不错过任何数据，CPU 必须每秒钟有 $1/(2 \times 10^{-3}) = 0.5 \times 10^3$ 次 DMA 传送，一秒钟内 CPU 用于硬盘 I/O 操作的时钟周期数为 $0.5 \times 10^3 \times (1000 + 500) = 750 \times 10^3$ 。因此，CPU 用于硬盘 I/O 的时间占整个 CPU 时间的百分比大约为 $750 \times 10^3 / (500 \times 10^6) = 1.5 \times 10^{-3} = 0.15\%$ 。

回顾例 8.2 中的中断控制 I/O 方式，其 CPU 用于硬盘 I/O 的占比为 25%，相比于 DMA 方式下的 0.15%，中断方式下，CPU 用于 I/O 的开销是 DMA 方式的 167 倍。 ■

DMA 方式下，数据传送不消耗任何处理器周期，所以即使硬盘一直在进行 I/O 操作，CPU 为它服务的时间也仅占 0.15%。事实上，硬盘在大多数时间内并不进行数据传送，而且每次 DMA 传输的数据块比 8KB 大得多，因此，CPU 为 I/O 所花费的时间会更少。当然，如果 CPU 同时也要访问存储器的话，由于存储器用于 DMA 传送，因此 CPU 会被延迟与存储器交换数据。但通过使用 cache，CPU 可避免大多数访存冲突，因为 CPU 的大部分访存过程都在 cache 中进行，所以存储器带宽的大部分都可让给 DMA 使用。

8.4 内核空间 I/O 软件

所有用户程序中提出的 I/O 请求，最终都是通过系统调用实现的，通过系统调用封装函数中的陷阱指令转入内核空间的 I/O 软件执行。内核空间的 I/O 软件分三个层次，分别是与设备无关的 I/O 软件层、设备驱动程序层和中断服务程序层，其中，后两个层次与 I/O 硬件密切相关。

8.4.1 与设备无关的 I/O 软件

一旦通过陷阱指令调出系统调用处理程序（如 Linux 中的 `system_call`）执行，就开始执行内核空间的 I/O 软件。首先执行的是与具体设备无关的 I/O 软件，主要完成所有设备公共的 I/O 功能，并向用户层软件提供一个统一的接口。通常，它包括以下几个部分：设备驱动程序统一接口、缓冲处理、错误报告、打开与关闭文件以及逻辑块大小处理等。

1. 设备驱动程序统一接口

对于某个外设具体的 I/O 操作，通常需要通过执行设备驱动程序来完成。而外设的种类繁多，控制接口不一致，导致不同外设的设备驱动程序千差万别。如果计算机系统中每次出现一种新的外设，都要为添加一种新设备驱动程序而修改操作系统，那么就会给操作系统开发者和系统用户带来很大的麻烦。

为此，操作系统为所有外设的设备驱动程序规定了一个统一的接口，新设备驱动程序只要按照统一的接口规范来编制，就可以在不修改操作系统的情况下，在系统中添加新设备驱动程序并使用新的外设进行 I/O。

因为采用了统一的设备驱动程序接口，因而，内核中与设备无关的 I/O 软件包含了所有外设统一的公共接口中的处理部分。例如，在图 8.3 所示的 Linux 系统调用函数执行过程中，刚陷入内核后所执行的 `system_call()` 函数就是与设备无关的 I/O 软件的一部分。

为了简化对外设的处理,在内核高层 I/O 软件中,将所有外设都抽象成一个文件,设备名和文件名在形式上没有任何差别,因而被称为设备文件名。内核中与设备无关的 I/O 软件必须将不同的设备名和文件名映射到对应的设备驱动程序。例如,在 UNIX/Linux 系统中,一个设备名将能够唯一地确定一个特殊文件的 i 节点。一个 i 节点中包含了主设备号,而主设备号可用于定位相应的设备驱动程序。i 节点还包括次设备号,次设备号可作为参数传递给设备驱动程序,用来确定进行 I/O 操作的具体设备位置。有关文件管理和设备管理的细节请参看操作系统方面的参考资料。

小贴士

在 UNIX/Linux 系统中,除了普通文件和目录文件以外,还有一类特殊文件,包括设备文件、链接文件等,设备特殊文件包含块设备特殊文件和字符设备特殊文件等,前者主要用于磁盘类设备,后者主要用于各类输入/输出设备,如终端、打印机、网络等。

i 节点是一个固定长度的表,包含对应文件相关的各种信息,如文件大小、文件所有者、文件访问权限,以及文件是普通文件、目录文件还是特殊文件等,在 i 节点中最重要的一项是磁盘地址列表。

特殊设备文件对应的 i 节点中包含主设备号和次设备号,主设备号确定设备类型(如 USB 设备、硬盘设备),因而它被系统用来确定设备驱动程序,次设备号被驱动程序用来确定具体的设备。

2. 缓冲区处理

用户进程在提出 I/O 请求时,指定的用来存放 I/O 数据的缓冲区在用户空间中。例如,文件读函数 `fread(buf, size, num, fp)` 中的缓冲区 `buf` 在用户空间中。通过陷阱指令陷入到内核态后,内核通常会在内核空间中再开辟一个或两个缓冲区,这样,在底层 I/O 软件控制设备进行 I/O 操作时,就直接使用内核空间中的缓冲区来存放 I/O 数据。为何不直接使用用户空间缓冲区呢?因为,如果直接使用用户空间缓冲区,那么,在外设进行 I/O 期间,由于用户进程被挂起而使用户空间的缓冲区所在页面有可能被替换出去,这样就无法获得缓冲区中的 I/O 数据。

每个设备的 I/O 都需要使用缓冲区,因而缓冲区的申请和管理等处理是所有设备公共的,可以包含在与设备无关的 I/O 软件部分。

此外,为了充分利用数据访问的局部性特点,操作系统通常在内核空间开辟高速缓存,将大多数最近从块设备读出或写入的数据保存在作为高速缓存的 RAM 区中。与设备无关的 I/O 软件会确定所请求的数据是否已经在高速缓存 RAM 中,如果存在的话,就可能不需要访问磁盘。

3. 错误报告

在用户进程中,通常要对所调用的 I/O 库函数返回的信息进行处理,有时返回的是错误码。例如, `fopen()` 函数的返回值为 `NULL` 时,表示无法打开指定文件。

虽然很多错误与特定设备相关,必须由对应的设备驱动程序来处理,但是,所有 I/O 操作在内核态执行时所发生的错误信息,都是通过与设备无关的 I/O 软件返回给用户进程的,也就

是说, 错误处理的框架是与设备无关的。

有些错误属于编程错误。例如, 请求了某个不可能的 I/O 操作; 写信息到一个输入设备或从一个输出设备读信息; 指定了一个无效的缓冲区地址或者参数; 指定了不存在的设备。这些错误信息由设备无关的 I/O 软件检测出来并直接返回给用户进程, 无需再进入底层的 I/O 软件处理。

还有一类是 I/O 操作错误。例如, 写一个已被破坏的磁盘扇区; 打印机缺纸; 读一个已关闭的设备。这些错误由相应的设备驱动程序检测出来并处理, 若驱动程序无法处理, 则驱动程序将错误信息返回给设备无关的 I/O 软件, 再由设备无关的 I/O 软件返回给用户进程。

4. 打开与关闭文件

对设备或文件进行打开或关闭等 I/O 函数所对应的系统调用, 并不涉及具体的 I/O 操作, 只要直接对 RAM 中的一些数据结构进行修改即可, 这部分工作也是由设备无关软件来处理。

5. 逻辑块大小的处理

为了为所有的块设备和所有的字符设备分别提供一个统一的抽象视图, 以隐藏不同块设备或不同字符设备之间的差异, 与设备无关的 I/O 软件为所有块设备或所有字符设备设置了统一的逻辑块大小。例如, 对于块设备, 不管磁盘扇区和光盘扇区有多大, 所有逻辑数据块的大小相同, 这样一来, 高层 I/O 软件就只需要处理简化的抽象设备, 从而在高层软件中简化了数据定位等处理。

8.4.2 设备驱动程序

设备驱动程序是与设备相关的 I/O 软件部分。每个设备驱动程序只处理一种外设或一类紧密相关的外设。每个外设或每类外设都有一个设备控制器, 其中包含各种 I/O 端口。通过执行设备驱动程序, CPU 可以向控制端口发送控制命令来启动外设, 可以从状态端口读取状态来了解外设或设备控制器的状态, 也可以从数据端口中读取数据或向数据端口发送数据等。显然, 设备驱动程序中包含了许多 I/O 指令, 通过执行 I/O 指令, CPU 可以访问设备控制器中的 I/O 端口, 从而控制外设的 I/O 操作。

根据设备所采用的 I/O 控制方式不同, 设备驱动程序的实现方式不同。

若采用程序直接控制 I/O 方式, 那么设备驱动程序将采用如图 8.12 所示的处理过程来控制外设的 I/O 操作, 驱动程序的执行与外设的 I/O 操作完全串行, 驱动程序一直等到全部完成用户程序的 I/O 请求后结束。驱动程序执行完成后, 返回到与用户无关的 I/O 软件, 最后, 再返回到用户进程。这种情况下, 用户进程在 I/O 过程中不会被阻塞, 内核空间的 I/O 软件一直代表用户进程在内核态进行 I/O 处理。

若采用中断控制 I/O 方式, 则设备驱动程序将采用如图 8.13 所示的处理过程来控制外设的 I/O 操作。驱动程序启动第一次 I/O 操作后, 将调用处理器调度程序来调出其他用户进程执行, 而请求 I/O 的用户进程 P 被阻塞。在 CPU 执行其他进程的同时, 外设进行 I/O 操作, 此时, CPU 和外设并行工作。当外设完成 I/O 任务时, 再向 CPU 提出中断请求, CPU 检测到中断请求后, 会暂停正在执行的其他进程, 转到如图 8.13b 所示的一个中断服务程序去执行, 以启动下一次 I/O 操作, 并在完成所有数据的 I/O 操作后解除用户进程 P 的阻塞状态。

若采用 DMA 控制 I/O 方式,那么,驱动程序将采用如图 8.18 所示的处理过程来控制外设的 I/O 操作。驱动程序对 DMA 控制器进行初始化后,便发送“启动 DMA 传送”命令,使设备控制器控制外设开始进行 I/O 操作,发送完启动命令后,将执行处理器调度程序,使 CPU 转去其他用户进程执行,而使请求 I/O 的用户进程 P 阻塞。DMA 控制器完成所有 I/O 任务后,向 CPU 发送一个“DMA 完成”中断请求信号。CPU 在中断服务程序中,解除用户进程 P 的阻塞状态,然后中断返回。

中断控制 I/O 和 DMA 控制 I/O 两种方式下,在执行设备驱动程序过程中都会进行处理器调度,以使当前用户进程阻塞;也都会产生中断请求信号,前者由设备在每完成一个数据的 I/O 后产生中断请求,后者由 DMA 控制器在完成整个数据块的 I/O 后产生中断请求。由此可见,中断请求是由于执行了设备驱动程序而产生的,外设完成驱动程序要求的 I/O 操作后,设备控制器或 DMA 控制器会向 CPU 发出中断请求,从而调出中断服务程序执行。

8.4.3 中断服务程序

图 8.19 给出了整个中断过程,包括两个阶段:中断响应和中断处理。中断响应是通过 CPU 执行中断隐指令来完成的,完全由硬件完成;而中断处理就是 CPU 执行一个中断服务程序的过程,完全由软件完成。虽然不同的中断源对应的中断服务程序不同,但是,所有中断服务程序的结构是相同的。中断服务程序包含三个阶段:准备阶段、处理阶段和恢复阶段。

图 8.19 给出的是多重中断系统下的中断服务程序结构。从图中可以看出,在保存断点、保护现场和旧的屏蔽字、设置新屏蔽字的过程中,CPU 一直处于“中断禁止”(“关中断”)状态。CPU 响应中断的第一件事就是关中断,即由 CPU 直接将中断允许触发器置 0,在进行具体的中断服务之前,再通过执行“开中断”指令来使中断允许触发器置 1,因此,在进行具体中断服务过程中,若有新的未被屏蔽的中断请求出现,则 CPU 可以响应新的中断请求。同样,在恢复阶段也要让 CPU 关中断,并在中断返回前开中断,这里开中断和关中断的功能都是通过 CPU 执行相应的“开中断”和“关中断”指令实现的。如果在准备阶段和恢复阶段 CPU 处在“开中断”状态,那么有可能在断点保存、现场和屏蔽字的保护和恢复等过程中响应新中断,这样,断点或现场及屏蔽字等重要信息就会被新中断破坏,因而不能回到原来的断点继续执行或因为现场或屏蔽字被破坏而不能正确执行。

图 8.19 中的“保护现场和旧屏蔽字”、“恢复现场和旧屏蔽字”分别通过“压栈”和“出栈”指令来实现,“设置新屏蔽字”和“清除中断请求”通过执行 I/O 指令

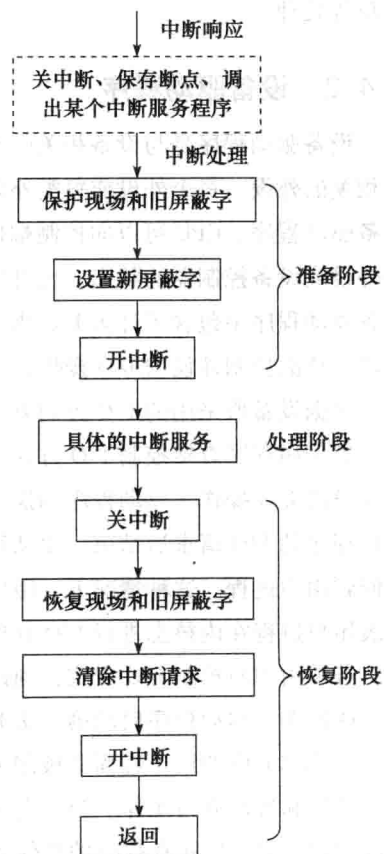


图 8.19 中断服务程序的典型结构

来实现, 这些 I/O 指令将对可编程中断控制器 (PIC) 中的中断请求寄存器和中断屏蔽字寄存器进行访问, 以使这些寄存器中相应的位清 0 或置 1。

在设备驱动程序和中断服务程序中使用到的 I/O 指令以及“开中断”和“关中断”指令都是特权指令, 只能在操作系统内核程序中使用。

例 8.4 在 IA-32/Linux 系统中, 假设某用户程序 P 中有以下一段 C 代码:

```
1 int len, n, buf[BUFSIZ];
2 FILE *fp;
3 ...
4 fp = fopen("bin_file.txt", "r");
5 n = fread(buf, sizeof(int), BUFSIZ, fp);
6 ...
```

假设文件 bin_file.txt 已经存在磁盘上且存有足够多的数据, 以前未被读取过。回答下列问题或完成下列任务。

① 执行第 4 行语句时, 从用户程序的执行到调出内核中的 I/O 软件执行的过程是怎样的? 要求画出函数之间的调用关系, 并用自然语言描述执行过程。

② 执行第 5 行语句时, 从用户程序的执行到调出内核中的 I/O 软件执行的过程是怎样的? 要求画出函数之间的调用关系。

③ 执行第 5 行语句时, 通过陷阱指令陷入内核后, 底层的内核 I/O 软件的大致处理过程是怎样的?

解 ① 执行第 4 行语句时, 从用户程序的执行到调出内核中的 I/O 软件执行的过程如图 8.20 所示。

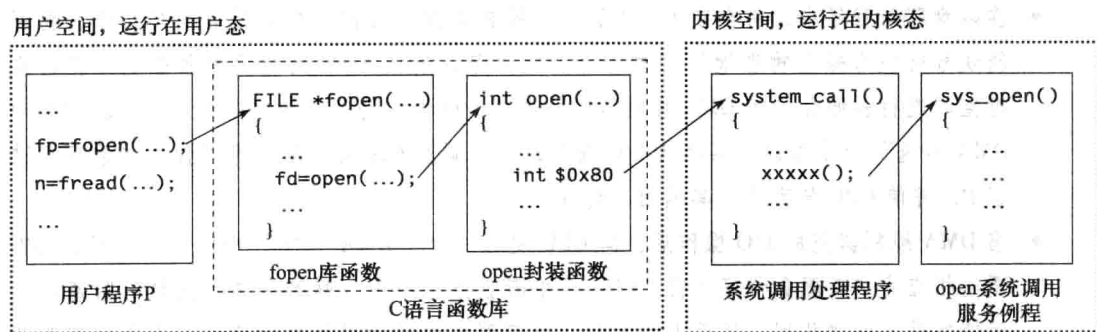


图 8.20 用户程序调用 fopen() 函数到内核 I/O 软件执行的过程

如图 8.20 所示, 当执行到用户程序 P 中第 4 行语句时, 将转入 C 标准库函数 fopen() 执行, 因为将要打开的文件 bin_file.txt 已经存在, 根据图 8.7 所示的 fopen() 函数源代码可知, fopen() 将调用系统调用函数 open(), 而 open 系统调用对应的指令序列中有一条陷阱指令 int \$0x80 (或 sysenter), 当执行到该陷阱指令时, 将从用户态陷入内核态执行, 在内核态首先执行的是 system_call 程序, 该程序中再根据系统调用号转到对应的 open 系统调用服务例程 sys_open 执行, 文件打开的具体工作由 sys_open() 完成。

② 执行第 5 行语句时, 从用户程序 P 的执行到调出内核中的 I/O 软件执行的过程如图 8.21 所示。

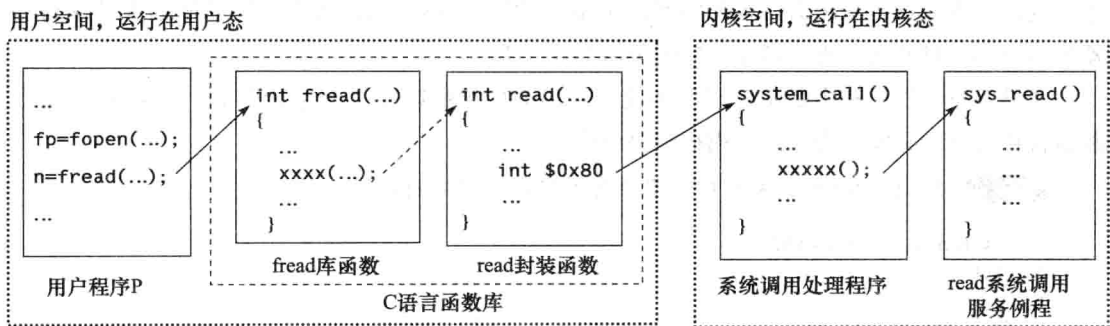


图 8.21 用户程序调用 fread() 函数到内核 I/O 软件执行的过程

③ 因为用户进程使用 `fread()` 函数读取的是一个普通的磁盘文件，所以应采用 DMA 控制 I/O 方式进行磁盘读操作。通过系统调用陷入内核后，底层的内核 I/O 软件的大致处理过程如下：

- 由内核空间中与设备无关的 I/O 软件完成以下相关操作：根据文件 `bin_file` 的文件描述符 `fd`（执行 `fopen()` 函数后得到一个指向结构 `FILE` 的指针 `fp`，在 `fp` 所指结构中包含了打开文件的文件描述符 `fd`），找到对应的文件描述信息，根据相应的文件描述信息可确定相应的磁盘设备驱动程序；根据文件当前指针确定所读数据在抽象的块设备中的逻辑块号；检查用户所需数据是否在高速缓存 RAM 中，以判断是否需要读磁盘。因为文件 `bin_file` 未曾被读取过，所以肯定不会在高速缓存 RAM 中，同时也不会用户在用户缓冲区，因而需要调用相应的磁盘驱动程序执行读磁盘操作。
- 在磁盘驱动程序中，将会完成以下操作：检查磁盘驱动器的电机是否运转正常，将逻辑块号转换为磁盘物理地址（柱面号、磁头号、扇区号），对将要接收磁盘数据的主存空间进行初始化，对 DMA 控制器中的各个 I/O 端口进行初始化；然后，发送“启动 DMA 传送”命令以启动具体的 I/O 操作；最后调用处理器调度程序以挂起当前用户进程 P，并使 CPU 转而执行其他用户进程。
- 当 DMA 控制器完成 I/O 操作后，向 CPU 发送一个“DMA 完成”中断请求信号，CPU 调出相应的中断服务程序执行。CPU 在中断服务程序中，解除用户进程 P 的阻塞状态而使其进入就绪队列，然后中断返回，再回到被打断的进程继续执行。下次处理器调度时用户进程 P 有可能会被调度到处理器上继续执行。 ■

8.5 小结

本章主要介绍与 I/O 操作相关的软件和硬件方面的相关内容。主要包括：文件的概念，I/O 系统调用函数，基本的 C 标准 I/O 库函数，常用外设控制器（I/O 接口）的基本功能和结构，I/O 端口的编址方式，外设与主机之间的 I/O 控制方式以及如何利用陷阱指令将用户 I/O 请求转换为操作系统的 I/O 处理过程。

用户程序通常通过调用编程语言提供的库函数或操作系统提供的 API 函数来实现 I/O 操

作, 这些函数最终都会调用系统调用的封装函数, 通过封装函数中的陷阱指令使用户进程从用户态转到内核态执行。

在内核态中执行的内核空间 I/O 软件主要包含三个层次, 分别是与设备无关的操作系统软件、设备驱动程序和中断服务程序。具体 I/O 操作是通过设备驱动程序或中断服务程序控制 I/O 硬件来实现的。设备驱动程序的实现主要取决于具体的 I/O 控制方式。

程序直接控制 I/O 方式下, 驱动程序实际上就是一个查询程序, 而且不再调中断服务程序。

中断控制 I/O 方式下, 驱动程序在启动完外设后, 将调用处理器调度程序以调出其他进程执行, 而使请求 I/O 的当前进程 P 阻塞; 当外设完成任务后, 则外设的设备控制器向 CPU 发出中断请求, CPU 调出中断服务程序执行; 在中断服务程序中, 进行新数据的读写或进行 I/O 操作的结束处理, 以解除用户进程 P 的阻塞状态。

DMA 控制 I/O 方式下, 驱动程序进行 DMA 传送初始化并发出“启动 DMA 传送”命令后, 将调用处理器调度程序以调出其他进程执行, 而使请求 I/O 的当前进程 P 阻塞; 当 DMA 传送完成后, 则 DMA 控制器向 CPU 发出“DMA 结束”中断请求, CPU 调出相应中断服务程序执行; 在中断服务程序中, 进行 DMA 结束处理, 包括解除用户进程 P 的阻塞状态。

在设备驱动程序和中断服务程序中, 通过执行 I/O 指令对设备控制器中的 I/O 端口进行访问。CPU 通过读取状态端口的状态来了解外设和设备控制器的状态, 根据状态向控制端口发送相应的控制信息, 以控制外设的读写和定位等操作, 而外设的数据则通过数据端口来访问。I/O 端口的编址方式有两种: 独立编址方式和统一编址 (存储器映射) 方式。

习题

1. 给出以下概念的解释说明。

I/O 硬件	I/O 软件	用户 I/O 软件	系统 I/O 软件
系统调用处理程序	系统调用服务例程	设备驱动程序	中断服务程序
流缓冲区	设备控制器	I/O 端口	控制端口
数据端口	状态端口	I/O 地址空间	独立编址
统一编址	存储器映射 I/O	I/O 指令	程序直接控制 I/O
就绪状态	中断控制 I/O	中断屏蔽字	中断请求寄存器
多重中断	中断嵌套	中断响应优先级	中断处理优先级
可编程中断控制器	DMA 控制 I/O 方式	DMA 控制器	设备无关的 I/O 软件

2. 简单回答下列问题。

- (1) I/O 子系统的层次结构是怎样的?
- (2) 系统调用的封装函数对应的机器级代码结构是怎样的?
- (3) 为什么系统调用的开销很大?
- (4) C 标准 I/O 库函数是在用户态执行还是在内核态执行?
- (5) 与 I/O 操作相关的系统调用函数是在用户态执行还是内核态执行?
- (6) I/O 端口的编址方式有哪两种? 各有何特点?

- (7) 什么是程序直接控制 I/O 方式? 说明其工作原理。
- (8) 什么是中断控制 I/O 方式? 说明其工作原理。
- (9) 为什么中断控制器把中断类型号放在 I/O 总线的数据线上而不是放在地址线上?
- (10) 为什么在保护现场和恢复现场的过程中, CPU 必须关中断?
- (11) DMA 控制 I/O 方式能够提高成批数据交换效率的主要原因何在?
- (12) DMA 控制器在什么情况下发出中断请求信号?

3. 以下是在 IA-32/Linux 系统中执行的用户程序 P 的汇编代码:

```

1  #hello.s #
2  #display a string "Hello, world."
3
4  .section .rodata
5  msg:
6  .ascii "Hello, world.\n"
7
8  .section .text
9  .globl _start
10 _start:
11
12 movl $4, %eax      #系统调用号(sys_write)
13 movl $1, %ebx      #file descriptor (参数一):文件描述符(stdout)
14 movl $msg, %ecx    #string address (参数二):要显示的字符串的首地址
15 movl $14, %edx     #string length (参数三):字符串长度
16 int $0x80          #调用内核功能
17
18 movl $1, %eax      #系统调用号(sys_exit)
19 movl $0, %ebx      #参数一:退出代码
20 int $0x80          #调用内核功能

```

针对上述汇编代码, 回答下列问题。

- (1) 程序的功能是什么?
 - (2) 执行到哪些指令时会发生从用户态转到内核态执行的情况?
 - (3) 该用户程序调用了哪些系统调用?
4. 第 3 题中用户程序的功能可以用以下 C 语言代码来实现:

```

1 int main()
2 {
3     write(1, "Hello, world.\n", 14);
4     exit(0);
5 }

```

针对上述 C 代码, 回答下列问题或完成下列任务。

- (1) 执行 write() 函数时, 传递给 write() 的实参在 main 栈帧中的存放情况怎样? 要求画图说明。
 - (2) 从执行 write() 函数开始到调出 write 系统调用服务例程 sys_write() 执行的过程中, 其函数调用关系是怎样的? 要求画图说明。
 - (3) 就程序设计的便捷性和灵活性以及程序执行性能等方面, 与第 3 题中的实现方式进行比较。
5. 第 3 题和第 4 题中用户程序的功能可以用以下 C 语言代码来实现:

```

1 #include <stdio.h>
2 int main()
3 {
4     printf("Hello, world.\n");
5 }

```

假定源程序文件名为 hello.c, 可重定位目标文件名为 hello.o, 可执行目标文件名为 hello, 程序用 GCC 编译驱动程序处理, 在 IA-32/Linux 系统中执行。回答下列问题或完成下列任务。

- (1) 为什么在 hello.c 的开头需加 “#include <stdio.h>”? 为什么 hello.c 中没有定义 printf() 函数, 也没它的原型声明, 但 main() 函数引用它时没有发生错误?
- (2) 需要经过哪些步骤才能在机器上执行 hello 程序? 要求详细说明各个环节的处理过程。
- (3) 为什么 printf() 函数中没有指定字符串的输出目的地, 但执行 hello 程序后会在屏幕上显示字符串?
- (4) 字符串 "Hello, world.\n" 在机器中对应的 0/1 序列 (机器码) 是什么? 这个 0/1 序列存放在 hello.o 文件的哪个节中? 这个 0/1 序列在可执行目标文件 hello 的哪个段中?
- (5) 若采用静态链接, 则需要用到 printf.o 模块来解析 hello.o 中的外部引用符号 printf, printf.o 模块在哪个静态库中? 静态链接后, printf.o 中的代码部分 (.text 节) 被映射到虚拟地址空间的哪个段中? 若采用动态链接, 则 printf 代码在虚拟地址空间中的何处?
- (6) 假定 printf() 函数最终调用的 write 系统调用的封装函数为 write(), 其对应的汇编代码如下:

804f8fa:	53	push	%ebx
804f8fb:	8b 54 24 10	mov	0x10(%esp), %edx
804f8ff:	8b 4c 24 0c	mov	0xc(%esp), %ecx
804f903:	8b 5c 24 08	mov	0x8(%esp), %ebx
804f907:	b8 04 00 00 00	mov	\$0x4, %eax
804f90c:	cd 80	int	\$0x80
804f90e:	5b	pop	%ebx
804f90f:	3d 01 f0 ff ff	cmp	\$0xffffffff, %eax
804f914:	0f 83 f6 1f 00 00	jae	8051910 <__syscall_error>
804f91a:	c3	ret	

请给出以上每条汇编指令的注释, 并说明该 Linux 系统中系统调用返回的最大错误号是多少?

- (7) 就程序设计的便捷性和灵活性以及程序执行性能等方面, 与第 3 题和第 4 题中的实现方式分别进行比较, 并分析说明哪个执行时间更短。
6. 假定采用中断控制 I/O 方式, 则以下各项工作是在 4 个 I/O 软件层的哪一层完成的?
 - (1) 根据逻辑块号计算磁盘物理地址 (柱面号、磁头号、扇区号)。
 - (2) 检查用户是否有权读写文件。
 - (3) 将二进制整数转换为 ASCII 码以便打印输出。
 - (4) CPU 向设备控制器写入控制命令 (如 “启动工作” 命令)。
 - (5) CPU 从设备控制器的数据端口读取数据。
7. 假设某计算机带有 20 个终端同时工作, 在运行用户程序的同时, 能接收来自任意一个终端输入的字符信息, 并将字符回送显示 (或打印)。每一个终端的键盘输入部分有一个数据缓冲寄存器 RDBR_i (i=1~20), 当在键盘上按下某一个键时, 相应的字符代码即进入 RDBR_i,

并使它的“完成”状态标志 $Done_i (i = 1 \sim 20)$ 置 1, 要等 CPU 把该字符代码取走后, $Done_i$ 标志才置 0。每个终端显示 (或打印) 输出部分也有一个数据缓冲寄存器 $TDBR_i (i = 1 \sim 20)$, 并有一个 $Ready_i (i = 1 \sim 20)$ 状态标志, 该状态标志为 1 时, 表示相应的 $TDBR_i$ 是空着的, 准备接收新的输出字符代码, 当 $TDBR_i$ 接收了一个字符代码后, $Ready_i$ 标志才置 0, 并将字符代码送到终端显示 (或打印), 为了接收终端的输入信息, CPU 为每个终端设计了一个指针 $PTR_i (i = 1 \sim 20)$ 指向为该终端保留的主存输入缓冲区。CPU 采用下列两种方案输入键盘代码, 同时回送显示 (或打印)。

- (1) 每隔一固定时间 T 转入一个状态检查程序 $DEVCHC$, 顺序地检查全部终端是否有任何键盘信息要输入, 如果有, 则顺序完成之。
- (2) 允许任何有键盘信息输入的终端向处理器发出中断请求。全部终端采用共同的向量地址, 利用它使处理器在响应中断后转入一个中断服务程序 $DEVINT$, 由后者询问各终端状态标志, 并为最先遇到的请求中断的终端服务, 然后转向用户程序。

要求画出 $DEVCHC$ 和 $DEVINT$ 两个程序的流程图。

8. 某台打印机每分钟最快打印 6 个页面, 页面规格为 50 行 $\times$ 80 字符。已知某计算机主频为 500MHz, 若采用中断控制 I/O 方式进行字符打印, 则每个字符申请一次中断且中断响应和中断处理时间合起来为 1000 个时钟周期。请问该计算机系统能否采用中断控制 I/O 方式进行字符打印输出? 为什么?
9. 假定某计算机的 CPU 主频为 500MHz, 所连接的某个外设的最大数据传输率为 20KB/s, 该外设接口中有一个 16 位的数据缓冲器, 相应的中断服务程序的执行时间为 500 个时钟周期, 则是否可以用中断控制 I/O 方式进行该外设的输入输出? 假定该外设的最大数据传输率改为 2MB/s, 则是否可以用中断控制 I/O 方式进行该外设的输入输出?
10. 若某计算机有 5 级中断, 中断响应优先级为 $1 > 2 > 3 > 4 > 5$, 而中断处理优先级为 $1 > 4 > 5 > 2 > 3$ 。要求:
 - (1) 设计各级中断处理程序的中断屏蔽位 (假设 1 为屏蔽, 0 为开放)。
 - (2) 若在运行主程序时同时出现第 2、4 级中断请求, 而在处理第 2 级中断过程中又同时出现 1、3、5 级中断请求, 试画出此程序运行过程示意图。
11. 假定某计算机字长 16 位, 没有 cache, 运算器一次定点加法时间等于 100ns, 配置的磁盘旋转速度为每分钟 3000 转, 每个磁道上记录两个数据块, 每一块有 8000 个字节, 两个数据块之间间隙的越过时间为 2ms, 主存周期为 500ns, 存储器总线宽度为 16 位, 总线带宽为 4MB/s, 假定磁盘采用 DMA 方式进行 I/O, CPU 时钟周期等于主存周期。回答下列问题。
 - (1) 磁盘读写数据时的最大数据传输率是多少?
 - (2) 当磁盘按最大数据传输率与主机交换数据时, 主存周期空闲百分比是多少?
 - (3) 直接寻址的“存储器-存储器”SS 型加法指令在无磁盘 I/O 操作干扰时的执行时间为多少? 当磁盘 I/O 操作与一连串这种 SS 型加法指令执行同时进行, 则这种 SS 型加法指令的最快和最慢执行时间各是多少?
12. 假设某计算机所有指令都在两个总线周期内完成, 一个总线周期用来取指令, 另一个总线周期用来存取数据。总线周期为 250ns, 因而每条指令的执行时间为 500ns。若该计算机中

配置的磁盘上每个磁道有 16 个 512 字节的扇区，磁盘旋转一圈的时间是 8.192ms，总线宽度 16 位，采用 DMA 方式传送磁盘数据，则在进行 DMA 传送时该计算机指令执行速度降低了百分之几？

13. 假设一个主频为 1GHz 的处理器需要从某个成块传送的 I/O 设备读取 1000 字节的数据到主存缓冲区中，该 I/O 设备一旦启动即按 50KB/s 的数据传输率向主机传送 1000 字节数据，每个字节的读取、处理并存入内存缓冲区需要 1000 个时钟周期，则以下几种方式下，在 1000 字节的读取过程中，CPU 花在该设备的 I/O 操作上的时间分别为多少？占整个处理器时间的百分比分别是多少？
- (1) 采用定时查询方式，每次处理一个字节，一次状态查询至少需要 60 个时钟周期。
 - (2) 采用独占查询方式，每次处理一个字节，一次状态查询至少需要 60 个时钟周期。
 - (3) 采用中断控制 I/O 方式，外设每准备好一个字节发送一次中断请求。每次中断响应需要 2 个时钟周期，中断服务程序的执行需要 1200 个时钟周期。
 - (4) 采用周期挪用 DMA 方式，每挪用一次主存周期处理一个字节，一次 DMA 传送完成 1000 字节数据的 I/O，DMA 初始化和后处理的时间为 2000 个时钟周期，CPU 和 DMA 没有访存冲突。
 - (5) 如果设备的速度提高到 5MB/s，则上述 4 种方式中，哪些是不可行的？为什么？对于可行的方式，计算出 CPU 花在该设备 I/O 操作上的时间占整个处理器时间的百分比？

数字逻辑电路基础

本附录主要介绍数字逻辑电路设计的基础知识,如果没有这些基础,可能会对本书中第2、5、6等章节中相关内容学习造成一定的障碍,因此,本附录的目的是为了帮助读者更好地理解本书正文中相关内容,而不是为了使读者获得逻辑电路设计能力。

本附录的主要内容包括:布尔代数、基本门电路、组合逻辑部件、时序逻辑部件和半导体存储器芯片。

A.1 布尔代数

冯·诺依曼结构计算机的一个重要特征是计算机中的信息采用二进制编码,也就是计算机中存储、运算和传送的信息都是二进制形式。因此,关于数字0和1的一套数学运算体系是非常重要的。这个体系起源于1850年前后英国数学家乔治·布尔(George Boole, 1815—1864)的工作,因此称为布尔代数。布尔注意到将TRUE(真)和FALSE(假)分别编码成1和0,可以通过设计一种代数运算系统来研究逻辑推理的基本原则。

布尔代数中只有两个数字0和1,分别表示逻辑值 FALSE 和 TRUE,因此,布尔代数是在二元集合 $\{0,1\}$ 基础上定义的。最基本的逻辑运算有三种:与(AND)、或(OR)、非(NOT), 逻辑运算符分别为“ $\cdot$ ”、“ $+$ ”和“ $-$ ”。图A.1给出了三种逻辑运算的定义。

A	B	$A \cdot B$	$A+B$	$\bar{A}$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

图 A.1 三种基本布尔代数运算

从图A.1可以看出, 与运算规则为:当且仅当输入值都为1时,结果为1,与运算也称为逻辑乘; 或运算规则为:只要输入值中有一个为1,结果就为1,或运算也称为逻辑加或逻辑和。 非运算也称为取反运算,规则为:输入值为1时结果为0,输入值为0时结果为1,非运算也称为逻辑反。

常见的布尔代数定律如下:

- ① 恒等定律: $A+0=A$, $A \cdot 1=A$ 。

② 0/1 定律: $A + 1 = 1$, $A \cdot 0 = 0$ 。

③ 互补律: $A + \bar{A} = 1$, $\bar{A} \cdot A = 0$ 。

④ 交换律: $A + B = B + A$, $A \cdot B = B \cdot A$ 。

⑤ 结合律: $A + (B + C) = (A + B) + C$, $A \cdot (B \cdot C) = (A \cdot B) \cdot C$ 。

⑥ 分配律: $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$, $A + (B \cdot C) = (A + B) \cdot (A + C)$ 。

逻辑表达式就是用逻辑运算符连接逻辑值而得到的表达式, 逻辑表达式的值仍为逻辑值。除了上述列出来的布尔代数定律, 还有其他重要的布尔代数定律, 如德摩根 (DeMorgan) 定律等, 利用这些布尔代数定律可以进行逻辑表达式的化简。

不难证明, 任何一种逻辑表达式都可以写成三种基本运算的逻辑组合。例如, **异或** (XOR) 运算的逻辑表达式为 $A \oplus B = \bar{A} \cdot B + A \cdot \bar{B}$ 。异或运算也称**不等价**运算。

上述逻辑表达式中的变量都是一个逻辑值, 即只有一位。很多时候可能处理的是一个由 n 个逻辑值组成的向量, 即按位逻辑运算。高级程序设计语言中都提供按位逻辑运算。例如, 对于 C 语言中的按位逻辑运算, 用 “|” 表示按位 “OR” 运算; 用 “&” 表示按位 “AND” 运算; 用 “~” 表示按位 “NOT” 运算; 用 “^” 表示按位 “XOR” 运算。

A.2 门电路

可以采用**逻辑门电路**来实现基本逻辑运算。例如, “**与门**”实现与操作, “**或门**”实现或操作, “**非门**” (也称为**反向器**) 实现取反操作。一个与门和一个或门可以有多个输入端, 但只有一个输出端, 而一个反向器只有一个输入端和一个输出端。图 A.2 给出了三种门电路的符号表示。

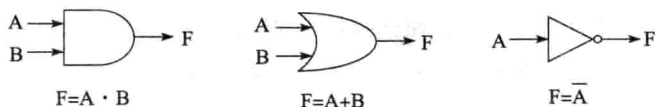


图 A.2 三种基本门电路符号

任何一种逻辑表达式都可以写成三种基本运算的逻辑组合, 因而任何一个逻辑电路都可以用与门、或门和非门的组合来实现。例如, 对于异或运算 $F = A \oplus B$, 可以用图 A.3 中所示电路来实现, 其中组合了两个非门、两个与门和一个或门。

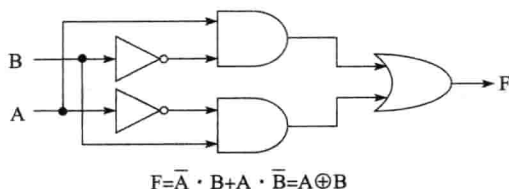
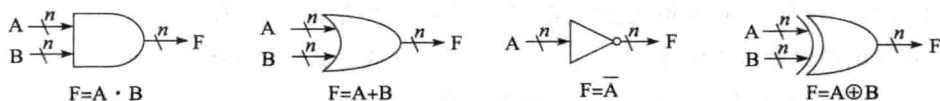


图 A.3 异或逻辑门电路的实现

在逻辑电路图中表示取反操作时, 通常都是在输入端或输出端上加一个 “○”。例如, 在图 A.3 中的或门输出端加一个 “○”, 则得到 $F = \overline{A \oplus B} = A \equiv B$, 即实现 “等价” 逻辑运算。

根据电路是否具有存储功能,将逻辑电路划分为两种类型:没有存储功能的电路称为组合逻辑电路,它的输出值仅依赖于当前输入值;含有存储功能的电路称为时序逻辑电路,它的输出值不仅依赖于当前输入值,还依赖于存储单元的当前状态。

上述门电路实现的是一位运算,如果是 n 位逻辑值的运算,只要重复使用 n 个相同的门电路即可,在逻辑电路图中无需画出所有的门电路,只要在输入端和输出端标注位数即可。例如,对于 n 位逻辑值 $A = A_{n-1}A_{n-2}\cdots A_1A_0$ 和 $B = B_{n-1}B_{n-2}\cdots B_1B_0$ 的与运算 $F = A \cdot B$,实际上是按位相与,即 $F_i = A_i \cdot B_i (0 \leq i \leq n-1)$ 。假定逻辑值的位数为 n ,则按位与、按位或、按位取反、按位异或的逻辑符号如图 A.4 所示。

图 A.4 n 位逻辑门电路符号

A.3 组合逻辑部件

可以利用基本逻辑门电路构成一些具有特定功能的组合逻辑部件,如译码器、编码器、多路选择器、加法器等。通常可以将一个功能部件的功能用一个真值表来描述,然后根据真值表确定逻辑表达式,最终根据逻辑表达式实现逻辑电路。真值表反映了功能部件的输入值和输出值之间的关系。

A.3.1 多路选择器

在第2章的图2.6中使用了一个多路选择器(MUX),有两个输入端和一个输出端,并有一个控制端,是二选一的多路选择器,每个输入端和输出端都有 n 位,因此,这样的多路选择器称为二路选择器。二路选择器如图 A.5 所示,其中图 A.5a 中是符号表示,图 A.5b 中给出一位二路选择器的门电路实现。

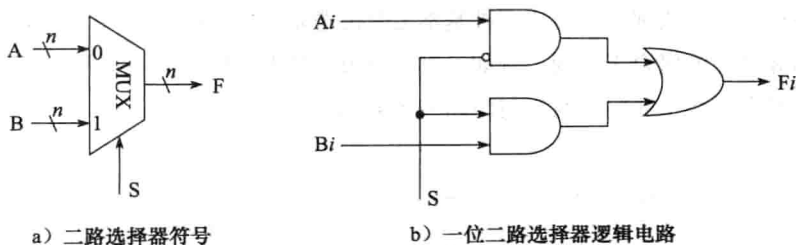


图 A.5 异或逻辑门电路的实现

从图 A.5 可以看出,一个二路选择器有一个控制端 S 、两个输入端 A 和 B 、一个输出端。其功能是:当 S 为0时, $F = A$;当 S 为1时, $F = B$ 。

推广到 k 路选择器,应该有 k 路输入,因而控制端 S 的位数应该是 $\lceil \log_2 k \rceil$ 。例如,对于3路或4路选择器, S 有两位;对于5~8路选择器, S 有3位。

A.3.2 无符号数加法器

在第2章的图2.6中使用了一个 n 位加法器，输入端有3个，分别是两个加数和一个低位来的进位；输出端有多个，除了和以外，还有向高位的进位以及多个标志，如溢出标志 OF、零标志 ZF 等。 n 位加法器是由 n 个一位加法器构成的。同时考虑两个加数和低位进位的一位加法器称为全加器 (Full Adder, 简称 FA)。全加器的真值表如图 A.6 所示。

如图 A.6 所示，全加器的两个加数为 A 和 B，低位进位为 C_{in} ，相加的和为 F，向高位的进位为 C_{out} 。根据真值表得到的全加器的逻辑表达式如下：

$$F = \bar{A} \cdot \bar{B} \cdot C_{in} + \bar{A} \cdot B \cdot \bar{C}_{in} + A \cdot \bar{B} \cdot \bar{C}_{in} + A \cdot B \cdot C_{in}$$

$$C_{out} = \bar{A} \cdot B \cdot C_{in} + A \cdot \bar{B} \cdot C_{in} + A \cdot B \cdot \bar{C}_{in} + A \cdot B \cdot C_{in}$$

使用布尔代数定律对上述逻辑表达式化简后得到全加和 F 和全加进位 C_{out} 的逻辑表达式为：

$$F = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

A	B	C_{in}	F	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

图 A.6 全加器真值表

根据全加器逻辑表达式，得到全加器逻辑电路如图 A.7 所示，其中图 A.7a 中是符号表示，图 A.7b 给出全加器的门电路实现。

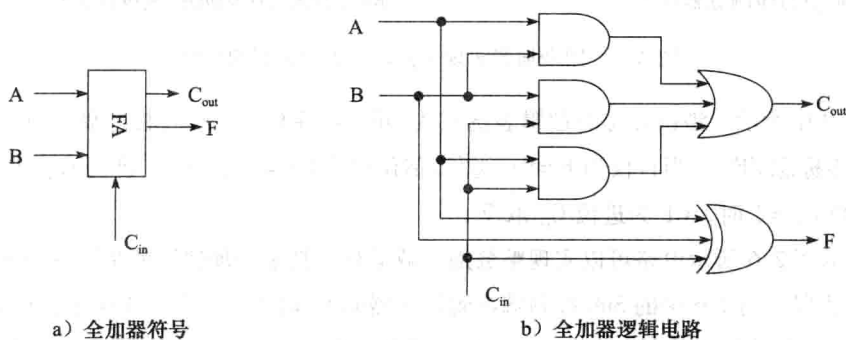


图 A.7 全加器逻辑门的实现

n 位无符号数加法器可由 n 个全加器实现，其电路如图 A.8 所示，其中图 A.8a 中是符号表示，图 A.8b 中给出用全加器构成加法器的实现电路， C_i 是第 $i-1$ 位向第 i 位的进位。

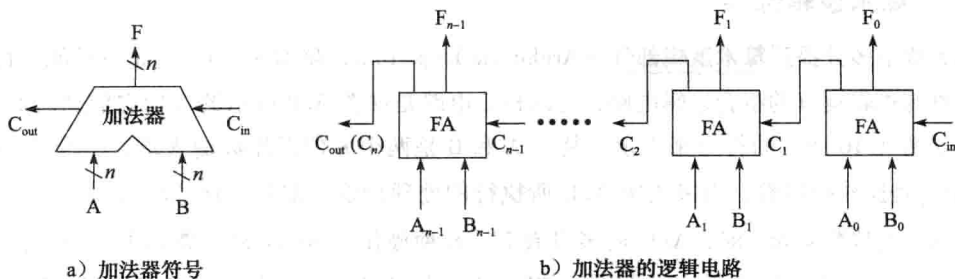


图 A.8 用全加器实现 n 位无符号数加法器的电路

用全加器实现 n 位无符号数加法器, 采用的是串行进位方式, 因而速度很慢。可以采用其他进位方式来实现快速加法器, 如采用先行进位方式的各类先行进位加法器等。

A. 3.3 整数加/减运算器

n 位无符号数加法器只能用于两个 n 位二进制数相加, 不能进行无符号整数的减运算, 也不能进行带符号整数的加/减运算。要能够进行无符号整数的加/减运算和带符号整数的加/减运算, 还需要在无符号数加法器的基础上增加相应的逻辑门电路, 使得加法器不仅能计算和/差, 还要能够生成相应的标志信息。图 A.9 是带标志信息的加法器实现电路示意图, 其中图 A.9a 中是符号表示, 图 A.9b 中给出用全加器构成的实现电路。

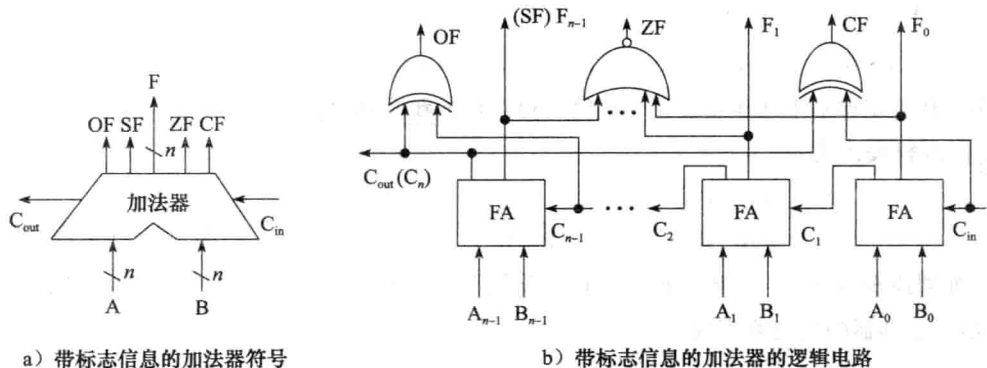


图 A.9 用全加器实现 n 位带标志加法器的电路

如图 A.9 中所示, 溢出标志的逻辑表达式为 $OF = C_n \oplus C_{n-1}$; 符号标志就是和的符号, 即 $SF = F_{n-1}$; 零标志 $ZF = 1$ 当且仅当 $F = 0$; 进位/借位标志 $CF = C_{out} \oplus C_{in}$, 即当 $C_{in} = 0$ 时, CF 为进位 C_{out} , 当 $C_{in} = 1$ 时, CF 为进位 C_{out} 取反。

第2章的图2.6所示电路可以实现整数加、减运算, 其中的加法器就是图A.9所示的带标志信息的加法器, 图2.6中的Sub控制端连到加法器的 C_{in} 输入端。在图2.6中, X 和 Y 是两个0/1序列, 对于带符号整数 x 和 y 来说, X 和 Y 就是 x 和 y 的补码表示, 对于无符号整数 x 和 y 来说, X 和 Y 就是 x 和 y 的无符号数表示。不管是带符号整数减法还是无符号整数减法, 都是用被减数加上减数的负数的补码来实现。

A. 3.4 算术逻辑部件

第5章中多处提到算术逻辑部件 (Arithmetic Logic Unit, 简称ALU), 它是一种能进行基本算术运算与逻辑运算的组合逻辑电路, 它的核心电路是如图A.9所示的带标志信息的加法器, 通常用如图A.10所示的符号来表示。其中 A 和 B 是两个 n 位操作数输入端, C_{in} 是进位输入端, ALU_{op} 是操作控制端, 用来决定ALU所执行的处理功能。 ALU_{op} 的位数 k 决定了操作的种类, 例如, 当位数 k 为3时, ALU最多只有 $2^3 = 8$ 种操作。Result是运算结果输出端, 此外, 还有相应的运算结果标志信息: 零标志ZF、溢出标志OF、符号标志SF和进位/借位标志CF等。

图 A. 11 给出了能够完成三种运算“与”、“或”和“加法”的一位 ALU 结构图。其中，一位加法用一个全加器实现，在 ALU_{op} 的控制下，由一个多路选择器（MUX）选择输出三种操作结果之一，因为 MUX 是三选一电路，所以 ALU_{op} 至少要有两位，即 $k=2$ 。当 $ALU_{op}=10(\text{add})$ 时，ALU 执行加法运算。

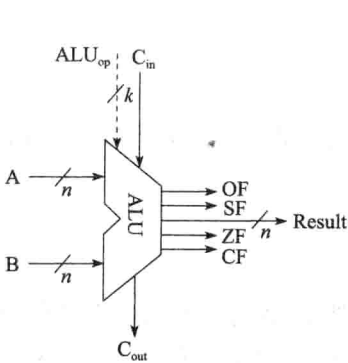


图 A. 10 ALU 符号

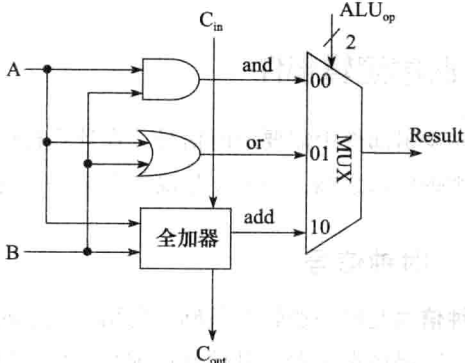
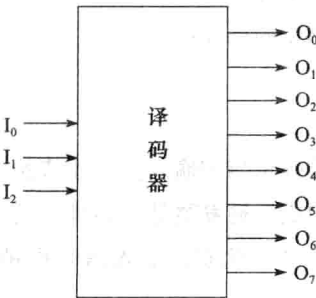


图 A. 11 一位 ALU 的结构

在一位 ALU 基础上，可以利用串行进位或单级、多级先行进位等方式构造 n 位 ALU，也可以在一个 n 位带标志信息的加法器基础上，加上其他逻辑电路，如 n 位与门、 n 位或门等电路实现 n 位 ALU。

A. 3.5 译码器

通常，若译码器的输入端有 n 位，则其输出端有 2^n 位。假定输入端分别为 $I_0, I_1, \dots, I_{n-1}$ ，输出端分别为 $O_0, O_1, \dots, O_{2^n-1}$ 。如果将输入的 n 位看成一个二进制数，则 n 位输入的可能取值为 $0, 1, 2, \dots, 2^n - 1$ 。若输入对应的二进制值为 i ，则输出端中只有 O_i 为 1，其他输出端全为 0。例如，当 $n=3$ 时，有 3 个输入端 I_0, I_1, I_2 ，8 个输出端 $O_0, O_1, \dots, O_7$ ，对应的译码器称为 3-8 译码器，其符号和真值表如图 A. 12 所示。



a) 3-8译码器符号

I_0	I_1	I_2	O_0	O_1	O_2	O_3	O_4	O_5	O_6	O_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

b) 3-8译码器真值表

图 A. 12 3-8 译码器的功能描述

译码器可以用来对地址码进行译码，例如，在第 6 章图 6. 1 关于主存储器组织中有地址译码器，它可对地址寄存器传送过来的地址进行译码，输出信号用来驱动地址码对应的地址选择

线对所在主存单元进行读写。同样,译码器也可以用于对指令的操作码进行译码,使不同的操作码能够得到不同的控制信号,以正确控制操作元件的动作。

与译码器功能相反的电路称为编码器,它的输入相当于译码器的输出,而输出相当于译码器的输入。

A.4 时序逻辑部件

时序逻辑部件中需要定时信号,在计算机中定时信号主要是时钟信号。因此,在讨论具体的时序逻辑电路之前,有必要先简单介绍一下时钟信号。

A.4.1 时钟信号

时钟信号是时序逻辑的基础,它用于确定时序逻辑元件中的状态在何时发生变化。时钟信号是由石英晶体谐振器与其他元件配合产生的具有固定周期的标准脉冲信号,如图 A.13 所示,每个时钟周期由高电平和低电平两个部分组成,时钟周期的倒数称为时钟频率。

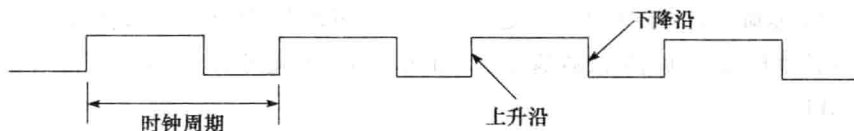


图 A.13 时钟信号

时序逻辑中状态的变化时点可以有不同的确定方式,本节仅讨论其中一种边沿触发方式。在边沿触发方式中,只有上升沿或下降沿是有效信号,即只有时钟边沿到达后才开始改变逻辑元件的状态。上升沿是指从低电平向高电平变化的边沿,而下降沿是指从高电平向低电平变化的边沿。在一个边沿触发的时序逻辑元件中,要么规定总是上升沿触发,要么规定总是下降沿触发,到底采用哪种边沿触发取决于逻辑设计技术,其设计思想是相同的。

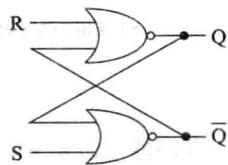
边沿触发方式就是将时钟信号的上升沿或者下降沿作为开始采样的同步点,每到一个采样同步点,就开始对时序逻辑元件中的状态进行采样,得到并存储新的采样信息。

A.4.2 触发器

含有存储功能的电路称为时序逻辑电路,它的输出值不仅依赖于当前输入值,还依赖于存储单元的当前状态。时序逻辑电路中的存储单元大多用触发器构成。触发器是一种具有两个稳定状态的电路,通过不同的输入信号可以将它设置成“0”或“1”状态,输入信号撤销后,它能保持原状态不变,因此具有记忆作用,能存储一位二进制信息。

最简单的存储单元是非时钟触发的电路,无需时钟信号输入。

图 A.14 是用一对交叉耦合的或非电路构成的存储单元,称为S-R 锁存器,也称为置位-重置锁存器。输出值 Q 是锁存器的状态值。其中, S 是置位 (Set) 输入端, R 是重置 (Reset) 输入端。当 S 端为 1 时,输出端 Q 变为 1,而下面的或非门则输出为 0,从而保持稳定的输出,图 A.14 S-R 锁存器



能起到锁存的作用；当 R 输入端为 1 时，则状态被重置，输出端 Q 变为 0，而下面的或非门输出为 1，进入另一个稳定状态；当 R 和 S 同时都变为 0 时，锁存器的状态保持不变；当 R 和 S 同时为 1 时，将产生不确定的结果，此时锁存器处于震荡或亚稳定状态。

时序逻辑电路中用到的存储单元大多是时钟信号触发的，其中最简单的存储单元是触发器和锁存器，它们与上述提到的 S-R 锁存器不同，都需要时钟作为触发信号。触发器和锁存器的差异在于导致状态变化的触发方式不同。在触发器中，它的状态变化仅发生在时钟边沿到来时刻，而在锁存器中，只要时钟信号有效，存储单元的输出会随着输入信号的变化而变化。

本书在第 5 章中提到的寄存器是由触发器构成的，而且在 5.2.2 节中提到了图 5.4 所示的 D 触发器的定时。为了使读者更好地理解触发器和寄存器的工作原理，本节主要介绍 D 触发器的内部结构和基本工作原理。

D 触发器可以用两个 D 锁存器构建，图 A.15 给出了利用或非门构建的 D 锁存器，它有两个输入端，其中，C 端是控制电路锁存信息的时钟信号，用于控制何时读取并存储输入端 D。当 C 有效时，锁存器处于“开状态”，输出端 Q 的值等于输入值 D；当 C 无效时，锁存器处于“关状态”，输出端 Q 不发生改变。图 A.16 给出了 D 锁存器的时序关系。

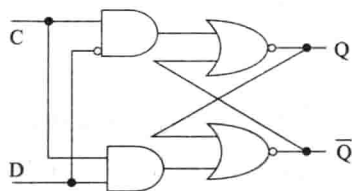


图 A.15 D 锁存器

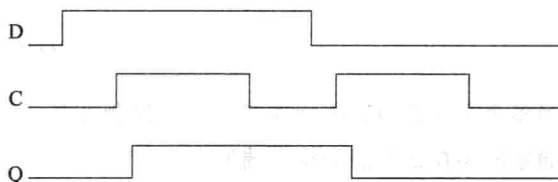


图 A.16 D 锁存器的时序图

图 A.17 所示的是下降沿触发的 D 触发器的内部结构，它与上升沿触发的 D 触发器没有本质差别，只要将图中的 C 端加一个反相器，将 C 取反后的信号送左边的 D 锁存器，再取反（即 C 的原码）送右边的 D 锁存器，就能构成上升沿触发的 D 触发器。图 A.18 给出了 D 触发器的时序关系。

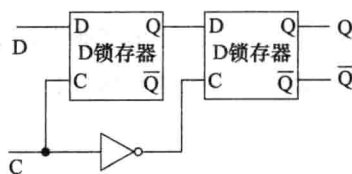


图 A.17 下降沿触发 D 触发器

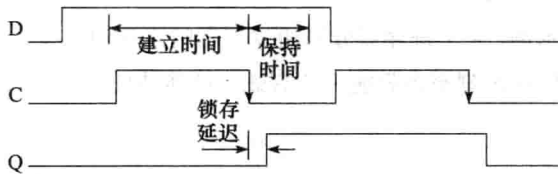


图 A.18 D 触发器的时序图

从图 A.17 所示的下降沿触发的 D 触发器的内部结构可以看出，当时钟信号 C 为高电平时，左边的 D 锁存器（称为主锁存器）处于开状态，输入端 D 被锁存在其 Q 端；当 C 从高电平变为低电平（下降沿）时，右边的 D 锁存器（称为从锁存器）处于开状态，将主锁存器 Q 端的值锁存起来，经过一段时间延迟，D 触发器的输出端 Q 就开始变为输入端 D 的值。

因为 D 触发器的输入端 D 在时钟触发边沿到来时被采样，因此，必须保证 D 的值在触发边沿前后的一定时间内保持不变，通常将时钟触发边沿到来之前的一段时间称为建立时间（setup time），将时钟触发边沿之后的一段时间称为保持时间（hold time）。也即，建立时间是

指在时钟触发边沿到来之前输入端 D 必须稳定有效的最短时间；保持时间是指在时钟触发边沿到来之后输入端 D 必须继续保持不变的最短时间。从时钟触发边沿到来时输出端 Q 改变为 D 的当前输入值的时间称为锁存延迟 (latch prop)，也称为Clk-to-Q 时间，这段时间远小于保持时间。

A.4.3 寄存器和寄存器堆

寄存器是用来暂存二进制数的逻辑部件，根据功能和实现方式的不同，有各种不同类型的寄存器。最简单的寄存器直接由若干个触发器组成。例如，由 n 个 D 触发器可构成一个 n 位寄存器，如图 A.19 所示。

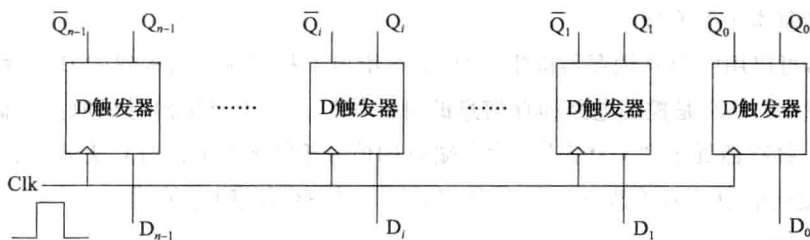


图 A.19 用 D 触发器构成 n 位寄存器

当多个寄存器与总线相连时，为了控制在某一时刻只有一个寄存器中的数据被送到总线上，通常需要在寄存器的输出端加上一个三态门，例如，第 5 章的图 5.7 和图 5.9 中在寄存器的输出端都加了三态门。

三态门 (three-state gate) 是一种重要的总线接口电路。三态指其输出既可以是通常的逻辑值 1 或逻辑值 0，又可以是特有的高阻抗状态。处于高阻抗状态时，输出电阻很大，没有任何逻辑控制功能，即相当于与所连接的总线是断开的。图 A.20 给出了三态门符号，三态门有一个输出使能端 EO，用于控制门电路与总线的接通与断开。当 EO 有效时，三态门的输出就是输入的值“0”或“1”；当 EO 无效时，三态门的输出为高阻态。图 A.21 给出了一个三态门运用示例，4 个寄存器分别通过三态门连接到总线，通过控制每个三态门的输出使能端，使它们在任何时刻至多只能一个有效，就能使任何时刻只有一个寄存器的内容被送到总线上。

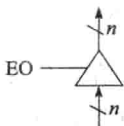


图 A.20 三态门符号

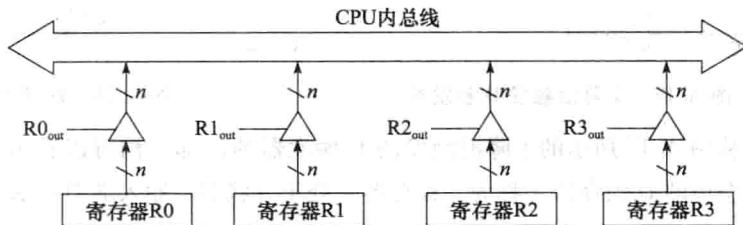


图 A.21 寄存器通过三态门与总线相连

在第 5 章介绍数据通路时提到 CPU 中有专门的寄存器堆 (register file)，用于暂存指令执行过程中用到的中间数据。寄存器堆也称为通用寄存器组 (General Register Set，简称 GRS)，它由许多寄存器组成，每个寄存器有一个编号，通过对寄存器编号进行译码，可以选择相应的

寄存器进行读写。图 A.22 是一个带时钟控制的双口寄存器堆的示意图, 有两个读口和一个写口, 每个读口或写口包括一个寄存器号输入端和一个读数据端或写数据端, 此外, 还有一个写使能输入端 WE, 它用来控制是否在下个时钟触发边沿到来时将 busW 线上的数据写入寄存器堆中。

图 A.22 所示寄存器堆中共有 2^k 个寄存器, 每个寄存器位数为 n , RA 和 RB 分别是读口 1 和读口 2 的寄存器编号, RW 是写口的寄存器编号。寄存器堆的读操作属于组合逻辑操作, 无需时钟控制, 即当寄存器地址 RA 或 RB 有效后, 经过一个“读取时间”的延迟, 在 busA 或 busB 上的信息开始有效。寄存器堆的写操作属于时序逻辑操作, 需要时钟信号的控制, 即在写使能信号 (WE) 有效的情况下, 下个时钟触发边沿到来时开始将 busW 上的信息写入 RW 所指的寄存器中。

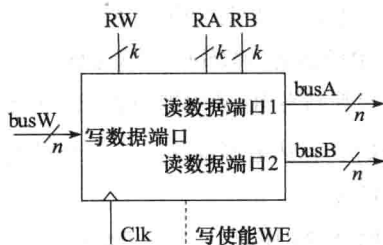


图 A.22 寄存器堆外部结构

A.5 半导体存储器芯片

第 6 章中多处提到半导体随机存储器芯片。半导体 RAM 具有体积小、存取速度快等优点, 因而适合作为内部存储器使用。按工艺不同可将半导体 RAM 分为双极型 RAM 和 MOS 型 RAM 两大类, MOS 型 RAM 又分为静态 RAM (Static RAM, 简称 SRAM) 和动态 RAM (Dynamic RAM, 简称 DRAM)。

A.5.1 基本存储元件

基本存储元件用来存储一位二进制信息, 是组成存储器的最基本的电路。下面介绍两种典型的分别用于 SRAM 芯片和 DRAM 芯片的存储元件。

1. 六管静态 MOS 管存储元件

六管静态 MOS 管存储元件如图 A.23 所示, 其中 T_1 和 T_2 构成触发器, T_3 、 T_6 是触发器的负载管, T_3 、 T_4 为门控管。使用这六个 MOS 管即可组成存储一位二进制信息的基本存储元。若 T_2 管导通, 则 T_1 管一定截止, 此时 A 点为高电平, B 点为低电平, 假定此时为存“1”状态; 反之 (当 T_1 管导通时) 则为存“0”状态。

(1) 信息的保持

字选择线 W 加低电平, T_3 与 T_4 截止, 触发器与外界隔离, 保持原有信息不变。

(2) 信息的读出

首先在两个位线上加高电平, 当字选择线 W 上加高电平时, T_3 与 T_4 开启。若原存“1”, 则 A

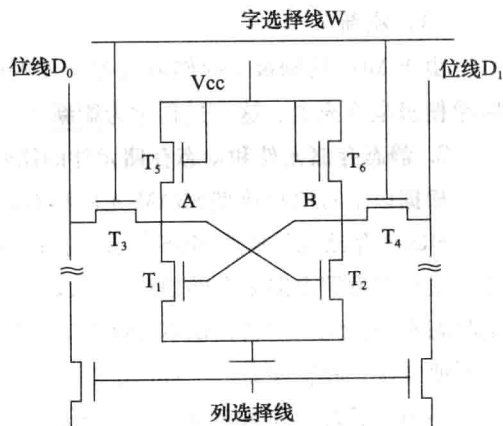


图 A.23 六管静态存储元件

点为高电平, T_2 导通, 此时有电流从位线 D_1 经 T_4 、 T_2 管流到地, 从而在位线 D_1 上产生一个负脉冲, 位线 D_0 上没有负脉冲。反之, 若原存“0”, 则在位线 D_0 上有负脉冲。根据哪条位线上有负脉冲可区分读出的是“0”还是“1”。

(3) 信息的写入

当字选择线 W 上加高电平时, T_3 与 T_4 开启。若要写“1”, 则在位线 D_1 上加低电平, 使 B 点电位下降, T_1 管截止, A 点电位上升, 使 T_2 管导通完成写“1”。若要写“0”, 则在 D_0 线上加低电平, 使 A 点电位下降, 结果使 T_2 管截止, B 点电位上升, T_1 管导通, 完成写“0”。

2. 单管动态 MOS 管存储元件

从六管静态 RAM 电路可看出, 即使存储元件不工作, 也有电流流过。例如, 当 T_1 导通, T_2 截止时, 有从电源经 $T_5 \rightarrow T_1 \rightarrow$ 地的电流流动。反之, 则有从电源经 $T_6 \rightarrow T_2 \rightarrow$ 地的电流流动, 因而功耗较大。

如图 A.24 所示, 动态 RAM 利用 MOS 管的栅极电容 C_s 来保存信息, 在信息保持状态下, 存储元件中没有电流流动, 因而大大降低了功耗。

DRAM 芯片中一般采用图 A.24 所示的单管动态单元电路, 其中 T 管为字选门控管, 读写时加选通脉冲使其导通。

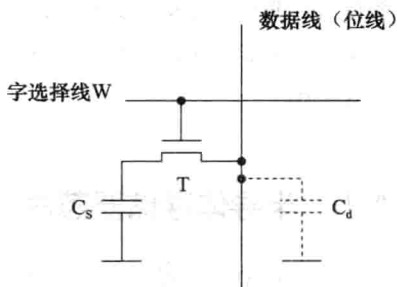


图 A.24 单管动态存储元件

(1) 信息的读出

若原存“1”, 则 C_s 上电荷通过 T 管在数据线上产生电流。反之, 若原存“0”, 则无电流。由此可区分读出的是 0 还是 1。因为读出时 C_s 上电荷放电, 电位下降, 所以是破坏性读出, 读后应有重写操作, 称为“再生”。由于 C_s 不可能很大, 所以 C_s 在数据线上放电产生的电流不会很大, 而且由于寄生电容 C_d 的存在, 放电时 C_s 上的电荷是在 C_s 和 C_d 之间分配, 因此, 读出电流值实际上非常小, 故对读出放大器的要求较高。

(2) 信息的写入

写“1”时, 在数据线上加高电平, 经 T 管对 C_s 充电; 写“0”则在数据线上加低电平, C_s 充分放电而使其上无电荷。

(3) 刷新

由于 MOS 管栅极上存储的电荷会缓慢放电, 超过一定时间, 就会丢失信息。因此必须定时给栅极电容充电, 这一过程称为刷新 (refresh)。

3. 静态存储元件和动态存储元件的比较

根据以上对两种典型 SRAM 元件和 DRAM 元件的介绍可看出它们各自的特点如下。

SRAM 存储元件所用 MOS 管多, 占硅片面积大, 因而功耗大, 集成度低; 但因为采用一个正负反馈触发器电路来存储信息, 所以, 只要直流供电电源一直加在电路上, 就能一直保持记忆状态不变, 无需刷新; 也不会因为读操作而使状态发生改变, 故无需读后再生; 特别是它的读写速度快, 其存储原理可看作是对带时钟的 RS 触发器的读写过程。由于 SRAM 价格比较昂贵, 因而, 适合做高速小容量的半导体存储器, 如 cache。

DRAM 存储元件所用 MOS 管少, 占硅片面积小, 因而功耗小, 集成度很高; 但因为采用

电容储存电荷来存储信息,会发生漏电现象,所以要使状态保持不变,必须定时刷新;因为读操作会使状态发生改变,故需读后再生;特别是它的读写速度相对 SRAM 元件要慢得多,其存储原理可看作是对电容充、放电的过程。相比于 SRAM, DRAM 价格较低,因而适合做慢速大容量的半导体存储器,如主存。

A.5.2 静态 RAM 芯片

如图 A.25 所示,静态 MOS 存储器芯片由存储体、I/O 读写电路、地址译码器和控制电路等部分组成。

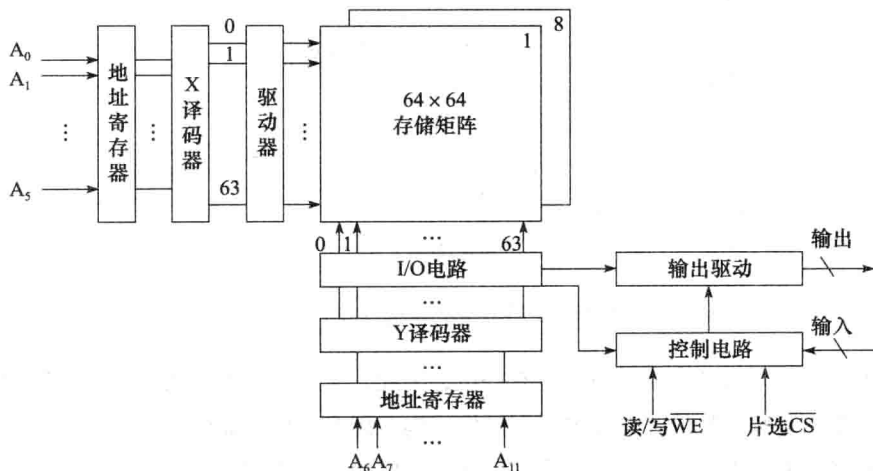


图 A.25 SRAM 存储器芯片结构图

① 存储体（存储矩阵）。存储体是存储单元的集合。如图 A.25 所示,4096 个存储单元被排成 64×64 的存储阵列,称为位平面,这样 8 个位平面构成 4096 字节的存储体。由 X 选择线（行选择线）和 Y 选择线（列选择线）来选择所需单元,不同位平面的相同行、列上的位同时被读出或写入。

② 地址译码器。用来将地址转换为译码输出线上的高电平,以便驱动相应的读写电路。地址译码有一维译码和二维译码两种方式。一维方式也称为线选法或单译码法,适用于小容量存储器;二维方式也称为重合法或双译码法,适用于容量较大的存储器。

在单译码方式下,只有一个行译码器,同一行中所有存储单元的字线连在一起,接到地址译码器的输出端,这样,选中一行中的各单元构成一个字,被同时读出或写入,这种结构的存储器芯片被称为字片式芯片。

地址位数较多时,地址译码器输出线太多。比如, $n=12$ 时单译码结构要求译码器有 4096 根输出线（字选择线）,因此大容量存储芯片不宜采用一维单译码方式。

目前,存储芯片大多采用双译码结构。地址译码器分为 X 和 Y 方向两个。图 A.25 采用的就是二维双译码结构,其对应的存储阵列组织如图 A.26 所示。

在图 A.26 所示的存储阵列中,有 4096 个单元,需要 12 根地址线 $A_0 \sim A_{11}$,其中 $A_0 \sim A_5$ 送至 X 译码器,有 64 条译码输出线,各选择一行单元; $A_6 \sim A_{11}$ 送至 Y 译码器,它也有 64 条

译码输出线, 分别控制一列单元的位线控制门。假如输入的 12 位地址为 $A_{11} A_{10} \cdots A_0 = 000000000001$, 则 X 译码器的第 2 根译码输出线 (x_1) 为高电平, 于是与它相连的 64 个存储单元的字选择 W 线为高电平。Y 译码器的第 1 根译码输出线 (y_0) 为高电平, 打开第一列的位线控制门。在 X、Y 译码的联合作用下, 存储矩阵中 (1, 0) 单元被选中。

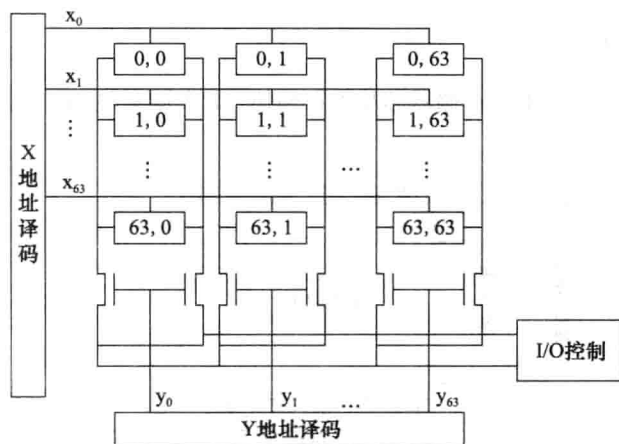


图 A.26 二维双译码结构（位片式芯片）

在选中的行和列交叉点上的单元只有一位, 因此, 采用二维双译码结构的存储器芯片被称为位片式芯片。有些芯片的存储阵列采用三维结构, 用多个位平面构成存储阵列, 不同位平面在同一行和列交叉点上的多位构成一个存储字, 被同时读出或写入。

③ 驱动器。在双译码结构中, 一条 X 方向的选择线要控制在其上的各个存储单元的字选择线, 所以负载较大, 因此需要在译码器输出后加驱动器。

④ I/O 控制电路。用以控制被选中的单元的读出或写入, 具有放大信息的作用。

⑤ 片选控制信号。单个芯片容量太小, 往往满足不了计算机对存储器容量的要求, 因此需将一定数量的芯片按特定方式连接成一个完整的存储器。在访问某个字时, 必须选中该字所在芯片, 而其他芯片不被选中。因而芯片上除了地址线 and 数据线外, 还应有片选控制信号。在地址选择时, 由芯片外的地址译码器的输入信号以及控制信号 (如“访存控制”信号) 来产生片选控制信号, 选中要访问的存储字所在的芯片。

⑥ 读/写控制信号。根据 CPU 给出的是读命令还是写命令, 控制被选中存储单元进行读或写。

A.5.3 动态 RAM 芯片

图 A.27 是典型的 $4M \times 4$ 位 DRAM 芯片示意图。DRAM 芯片容量较大, 因而地址位数较多, 为了减少芯片的地址引脚数, 从而减小体积, 大多采用地址引脚复用技术, 行地址和列地址通过相同的引脚分先后两次输入, 这样地址引脚数可减少一半。

图 A.27a 给出了芯片的引脚, 共有 11 根地址引脚线 $A_0 \sim A_{10}$, 在行选通信号 $\overline{RAS}$ 和列选通信号 $\overline{CAS}$ 的控制下分时传送行、列地址; 有 4 根数据引脚线 $D_1 \sim D_4$, 因此, 每个芯片同时读出

4 位数据； $\overline{\text{WE}}$ 为读写控制引脚，低电平时为写操作； $\overline{\text{OE}}$ 为输出使能驱动引脚，低电平有效，高电平时断开输出，即输出呈高阻态。

图 A. 27b 给出了芯片内部的逻辑结构图，芯片存储阵列采用三维结构，芯片容量为 $2048 \times 2048 \times 4$ 位，因此，行地址和列地址各 11 位，有 4 个位平面，在每个行、列交叉处的 4 个位平面数据同时进行读写。行地址缓冲器和刷新计数器通过一个多路选择器（MUX）将选择的行地址输出到行译码器，刷新计数器的位数也是 11 位，一次刷新相当于对一行数据进行一次读操作，每次读后需再生。

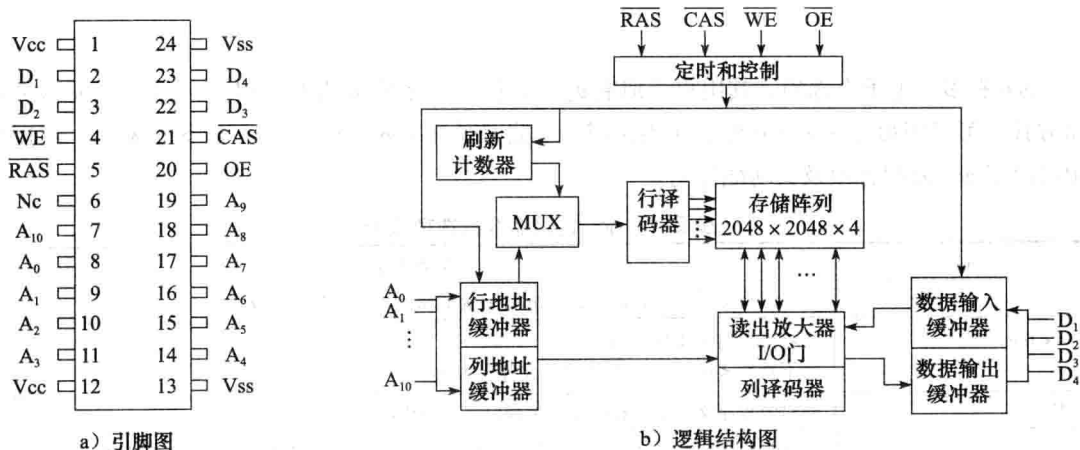


图 A. 27 4M x 4 位 DRAM 芯片

gcc 的常用命令行选项

gcc 有多达上千个选项，其用户手册有近一万行，大多数选项很少用到，想了解 gcc 的使用方式，可以用命令 `man gcc` 显示使用说明。本附录给出 gcc 常用的几个命令行选项。表 B.1 中给出了 gcc 常用选项及其功能说明。

表 B.1 gcc 常用命令行选项说明

选项	功能描述
-c	只进行编译不进行链接，生成以 .o 为后缀的可重定位目标文件
-o <file> -o	将结果写入文件 <file> 中 不指定 <file> 时，默认结果文件名为 a.out
-E	对源程序文件进行预处理，生成以 .i 为后缀的预处理文件
-S	对源程序文件或预处理文件进行汇编，生成以 .s 为后缀的汇编语言目标文件
-v	在标准错误输出上输出编译过程中执行的命令及程序版本号
-w	不输出任何警告级错误信息
-Wall	在标准错误输出上输出所有可选的警告级错误信息
-g	生成调试辅助信息，以便使用 GDB 等调试工具对程序进行调试
-pg	编译时加入剖析代码，以产生供 gprof 剖析用的统计信息
-O -O <n>	指定编译优化级别，<n> 可以是 0、1、2、3 或者 s，-O 或缺省该选项时都为 -O1。 -O0 表示不进行优化；-O3 的优化级别最高；-Os 相当于 -O2.5，表示使用所有不会增加代码量的二级优化（-O2）
-D <name> -D <name> = <def>	-D <name> 将宏 <name> 默认定义为 1 显式地定义宏 <name> 等于 <def>
-I <dir>	将目录 <dir> 加入到头文件的搜索目录集合中，链接时在搜索标准头文件之前先对 <dir> 进行搜索
-L <dir>	将目录 <dir> 加入到库文件的搜索目录集合中，链接时在搜索标准库文件之前先对 <dir> 进行搜索

小贴士

gprof 是 GNU 工具之一，它是一个剖析程序（profiler），通过编译时在每个函数的出入口加入剖析代码，来监控程序在用户态的执行信息，可以得到每个函数的调用次数、执行时间、调用关系等信息，从而便于程序员查找用户程序的性能瓶颈。但是，对于很多时间都在内核态执行的程序，并不适合用 gprof 进行剖析。

有些剖析工具可以对内核运行情况进行剖析。例如，oprofile 是一个开源的剖析工具，它使用处理器中的性能监视硬件来监控关于内核以及可执行文件的执行信息，监控开销比较小，而且统计信息较多，可以统计诸如 cache 缺失率、主存访问信息、分支预测错误率等，这些信息使用 gprof 是无法得到的，不过，oprofile 不能得到函数调用次数。

总之，gprof 工具较简单，适合于查找用户程序的瓶颈，而 opprofile 工具稍显复杂，能得到更多性能方面的信息，更适合剖析系统软件。

GDB 的常用命令

GDB 是一个程序调试工具软件。GDB 中有一个非常有用的命令补齐功能，如同 Linux 下 shell 命令解释器中的命令补齐功能一样，输入一个命令的前几个字符后再输入 TAB 键时，能补齐命令。如果有多个命令的前几个字符相同，则会发出警告声，再输入 TAB 键后，则将所有前几个字符相同的命令全部列出。

C.1 启动 GDB 程序

可以在 shell 命令行提示符下输入“gdb”命令来启动 GDB 程序。假定 shell 命令行提示符为“unix>”，最常用的启动 GDB 程序的方式如下：

```
unix> gdb [可执行文件名]
```

该命令用于启动 GDB 程序并同时加载指定的将要被调试的可执行文件。如果仅输入“gdb”而没有带可执行文件名，则仅启动 GDB 程序，因此必须在 GDB 调试环境下再通过相应的 GDB 命令来加载需调试的可执行文件。

一旦启动 GDB 程序，则调试过程就在 GDB 调试环境下进行。

C.2 常用 GDB 命令

在 GDB 调试环境下，大部分 GDB 命令都可以利用补齐功能以简便方式输入。如 quit 可以简写为 q，因为以 q 打头的命令只有 quit。list 可以简写为 l 等。此外，按回车键将重复上一个命令。

在 GDB 调试环境下使用的 GDB 命令有很多，本附录仅介绍最常用的几个。

- **help** [命令名]

若想了解某个 GDB 命令的用法，最方便的方法是使用 help 命令。例如，在 gdb 提示符下输入 help list 将显示 list 命令的用法。

- **file** <可执行文件名>

如果在启动 GDB 程序时忘记加可执行文件名，则在调试环境下可用 file 命令指定需加载并调试的可执行文件。例如，可用命令“file./hello”加载当前目录下的 hello 程

序。注意，可执行文件的路径名一定要正确。

- **run** [参数列表]

run 命令用来启动并运行已加载的被调试程序，如果被调试程序需要参数，则在 **run** 后接着输入参数列表，参数之间用空格隔开。

- **list** [显示对象]

list 命令用来显示一段源程序代码。在 **list** 后面指定显示对象的参数通常有以下几种。

< **linenum** > : 行号，显示对象为指定行号前、后若干行源码。

< + **offset** > : 相对当前行的正偏移量，显示对象为当前行的后面若干行源码。

< - **offset** > : 相对当前行的负偏移量，显示对象为当前行的前面若干行源码。

< **filename** : **linenum** > : 显示对象为指定文件中指定行号前、后若干行源码。

< **function** > : 函数名，显示对象为指定函数的源码。

< **filename** : **function** > : 显示对象为指定文件中指定函数的源码。

< * **address** > : 地址，显示指定地址处的源码。

- **break** [需设置的断点]

break 命令用来对被调试程序设置断点。在 **break** 后面的参数通常有以下几种。

< **linenum** > : 行号，在当前源文件中的指定行处设置断点。

< **filename** : **linenum** > : 在指定文件的指定行处设置断点。

< **function** > : 函数名，在指定函数的入口处设置断点。

< **filename** : **function** > : 在指定文件中指定函数的入口处设置断点。

< * **address** > : 地址，在指定地址处设置断点。

< **condition** > : 条件，只有在某些特定的条件成立时程序才会停下，称为条件断点。

设置一个断点后，它的起始状态是有效。可以用 **enable**、**disable** 来使某断点有效或无效，也可以用 **delete** 命令删除某断点。例如，可以用命令“**disable 2**”使 2 号断点无效，用“**delete 2**”删除 2 号断点。

- **info br** | **source** | **stack** | **args** | ...

info 命令用来查看被调试程序的信息，其参数非常多，但大部分不常用。其中，**info br** 查看设置的所有断点的详细信息，包括断点号、类型、状态、内存地址、断点在源程序中的位置等；**info source** 查看当前源程序；**info stack** 查看栈信息，它反映了过程（函数）之间的调用层次关系；**info args** 查看当前参数信息。

- **watch** <表达式>

watch 命令用来观察某个表达式或变量的值是否被修改，一旦修改则暂停程序执行。

- **print** <表达式>

print 命令用来显示表达式的值，表达式中的变量必须是全局变量或当前栈区可见的变量。否则 GDB 会显示以下类似信息：

No symbol "xxxxx" in current context.

- **x /NFU address**

x 命令用来检查内存单元的值，x 是 examine 的意思。其中 N 代表重复数，F 代表输出格式，U 代表每个数据单位的大小，上述命令表示从地址 address 开始以 F 格式显示 N 个大小为 U 的数值。若不指定 N，则默认为 1；若不指定 U，则默认为每个数据单位为 4 个字节。F 的取值可以是 x(十六进制整数)、d(带符号十进制整数)、u(无符号十进制整数)或 f(浮点数格式)；U 的取值可以是 b(字节)、h(双字节)、w(四字节)或 g(八字节)。例如，命令 x/8ub 0x8049000 表示如下含义：以无符号十进制整数格式(u)显示 8 个字节(b)，即显示存储单元 0x8049000、0x8049001、0x8049002 和 0x8049003 中的内容。

- **step**

使用 step 命令可以跟踪进入到一个函数的内部。

- **next**

使用 next 命令继续执行下一条语句，若当前语句中包含函数调用，则不会进入函数内部，而是完成对当前语句中的函数调用后跟踪到下一条语句。

- **continue**

当程序在断点处暂停执行后，可以用 continue 命令使程序继续执行下去。

- **quit**

使用 quit 命令可退出 GDB。

参考文献

- [1] Randal E Bryant, David R. O'Hallaron. 深入理解计算机系统 [M]. 龚奕利, 雷迎春, 译. 2 版. 北京: 机械出版社, 2011.
- [2] 袁春风. 计算机组成与系统结构 [M]. 北京: 清华大学出版社, 2010.
- [3] Daniel P Bovet, Marco Cesati. 深入理解 LINUX 内核 [M]. 陈莉君, 张琼声, 张宏伟, 译. 3 版. 北京: 中国电力出版社, 2007.
- [4] 新设计团队. Linux 内核设计的艺术 [M]. 北京: 机械工业出版社, 2013.
- [5] Robert Love. Linux 内核设计与实现 [M]. 陈莉君, 康华, 译. 3 版. 北京: 机械工业出版社, 2011.
- [6] Marc J Rochkind. 高级 UNIX 编程 [M]. 王嘉祯, 杨素敏, 张斌, 等译. 2 版. 北京: 机械工业出版社, 2006.
- [7] Johnson M Hart. Windows 系统编程 [M]. 戴峰, 陈征, 等译. 4 版. 北京: 机械工业出版社, 2010.
- [8] Brian W Kernighan, Dennis M Ritchie. The C Programming Language [M]. 2 版. 北京: 机械工业出版社, 2006.
- [9] 尹宝林. C 程序设计导引 [M]. 北京: 机械工业出版社, 2013.
- [10] Andrew S Tanenbaum. 现代操作系统 [M]. 陈向群, 马洪兵, 等译. 2 版. 北京: 机械工业出版社, 2008.
- [11] 教育部考试中心. 全国计算机等级考试 三级教程——PC 技术 [M]. 北京: 高等教育出版社, 2009.